

24CSI011 - COMPUTER NETWORKS AND SECURITY



UNIT 5 NETWORKS SECURITY CONCEPTS

Dr.N.SUGANTHI

Department of Computer Science and
Engineering



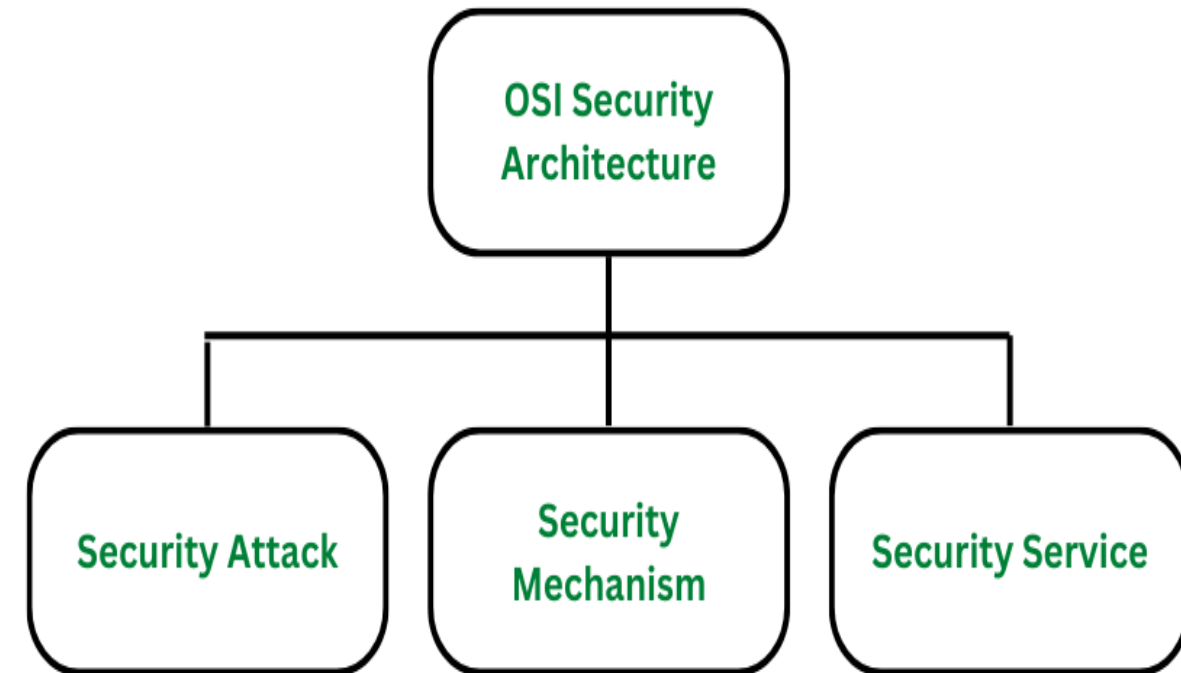
OSI Security Architecture

- ITU-T X.800 Security Architecture for OSI defines a systematic way of defining and providing security requirements
- It provides a useful, if abstract, overview of concepts,

OSI Security Architecture

Focus on the following:

- **Security attack**
 - **Passive attack – unauthorized reading of a message and traffic analysis**
 - **Active attack – modification of messages**
- **Security mechanism**
 - **Any process that is designed to detect, prevent or recover from a security attack. (e.g) encryption algorithms, digital signatures.**
- **Security service**
 - **Authentication, access control , data confidentiality, data integrity, non-repudiation.**



Security Service

Security service is something that **enhances the security of the data processing systems and the information transfers** of an organization

Intended to **counter security attacks**

Make use of one or more security mechanisms to provide the service

Replicate functions normally associated with physical documents

- eg. have signatures, dates; need protection from disclosure, tampering, or destruction; be notarized or witnessed; be recorded or licensed

Security Mechanism



It is a mechanism that is **designed to detect, prevent, or recover from a security attack.**



No single mechanism that will support all functions required



However one particular element underlies many of the security mechanisms in use: **cryptographic techniques**



Hence our focus on this area

Security Attack



Any action that **compromises the security of information** owned by an organization



Information security is about how to prevent attacks, or failing that, to detect attacks on information-based systems



Have a wide range of attacks



Can focus of generic types of attacks



Note: often *threat* & *attack* mean same

Security Attacks

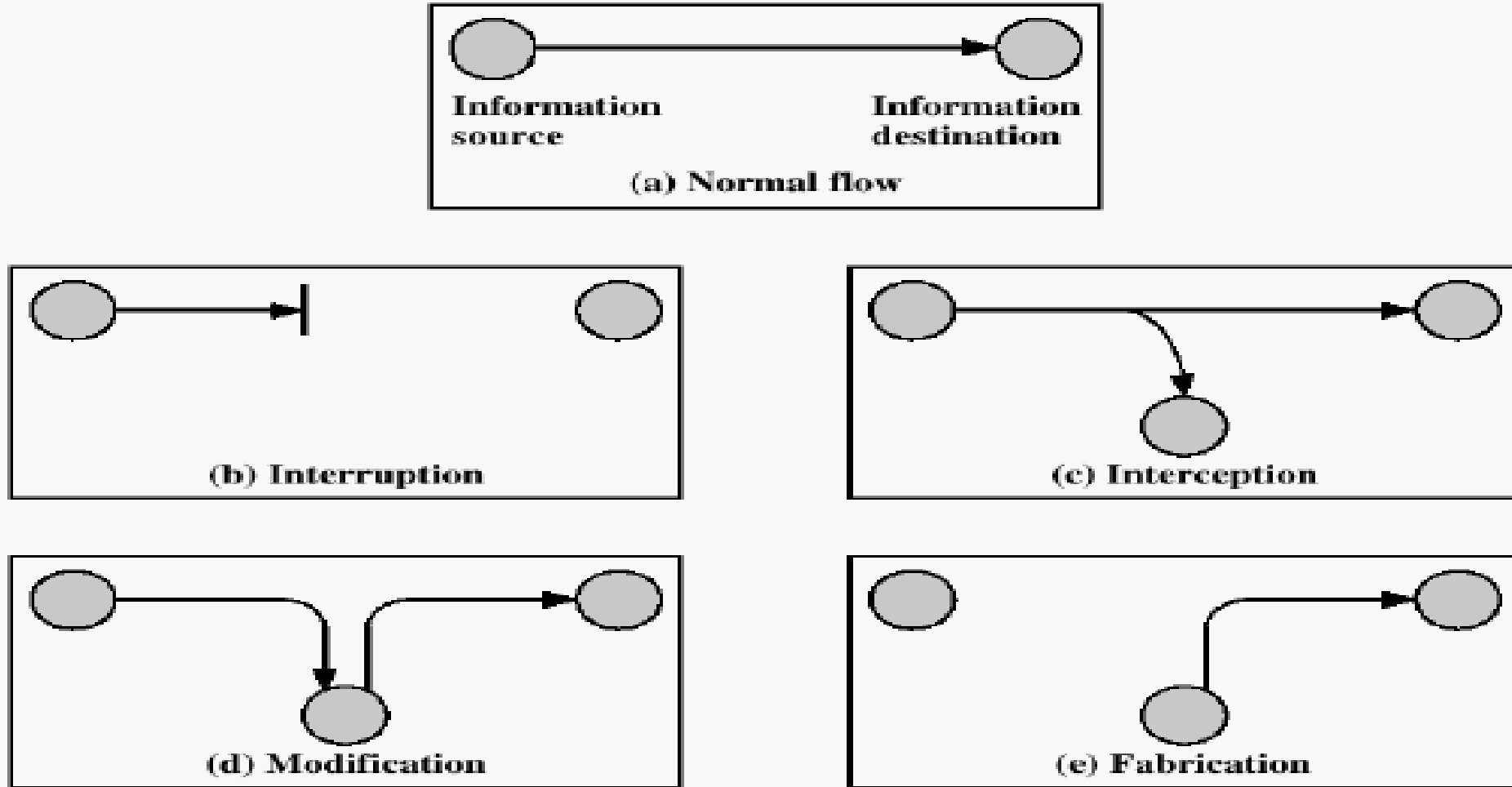


Figure 1.1 Security Threats

Security Attack



Passive: A passive attack attempts to learn or make use of information from the system but **does not affect system resources**

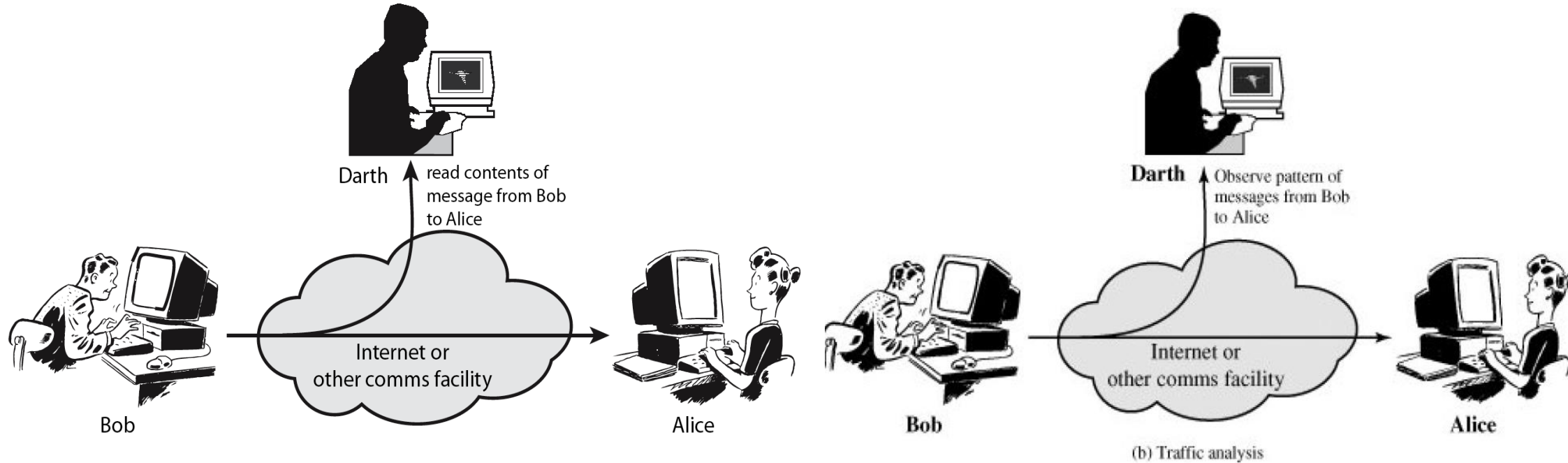


Active : An active attack attempts to **alter system resources** or affect their operation.

Passive Attacks

Traffic Analysis:

- Location
- Identity of communicating hosts,
- Frequency and length of the message



By eavesdropping on, or monitoring of, transmissions to:

I. Obtain message contents

II. Monitor traffic flows

Are difficult to detect because they do not involve any alteration of the data.

Emphasis is on **preventing passive attack(encryption)** rather than detection.

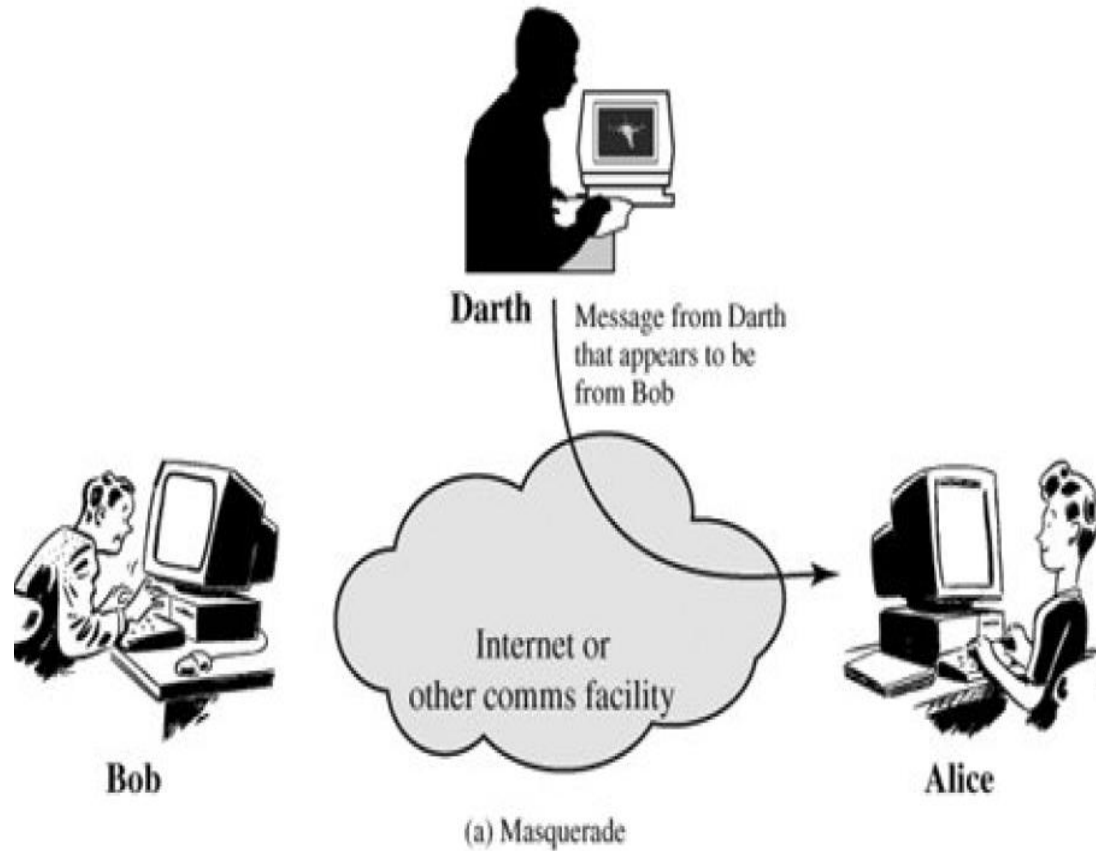
Active Attacks

Involve some **modification of data stream or creation of a false stream**.
Subdivided into four categories:

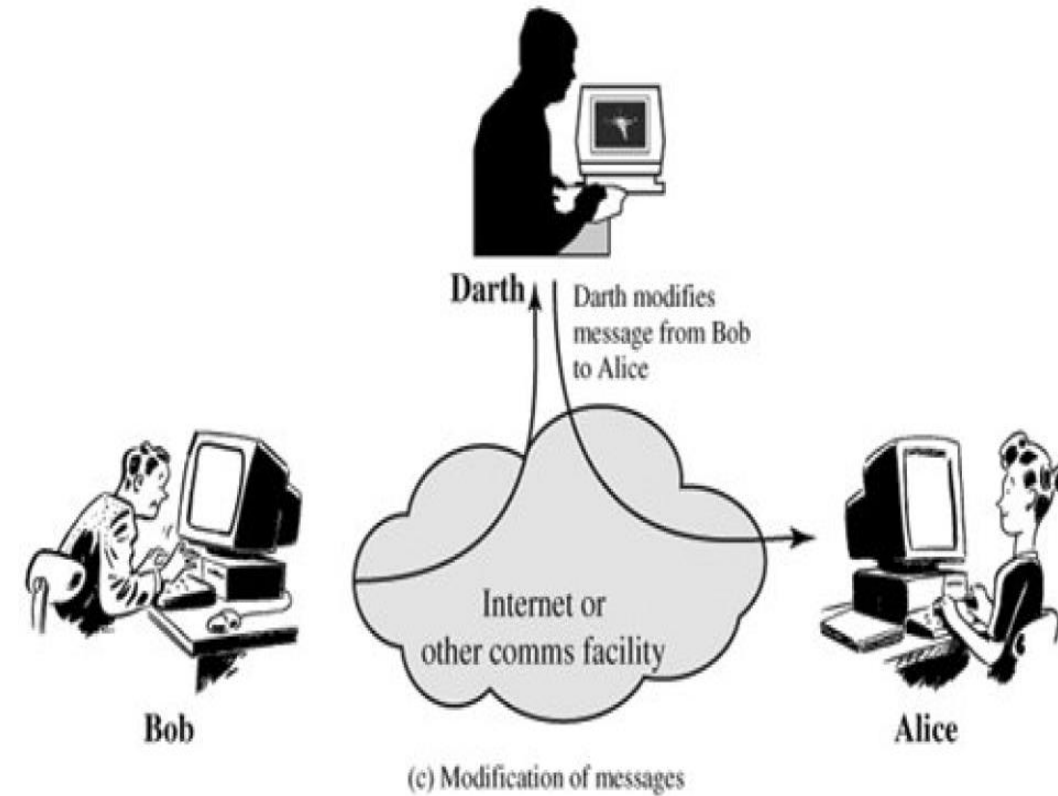
- I. Masquerade of one entity as some other**
- II. Replay previous messages**
- III. Modify messages in transit**
- IV. Denial of service**

- Goal is to **detect active attacks**.
- **Difficult to prevent active attacks** due to wide variety of potential physical, software, hardware vulnerabilities.

Active Attacks

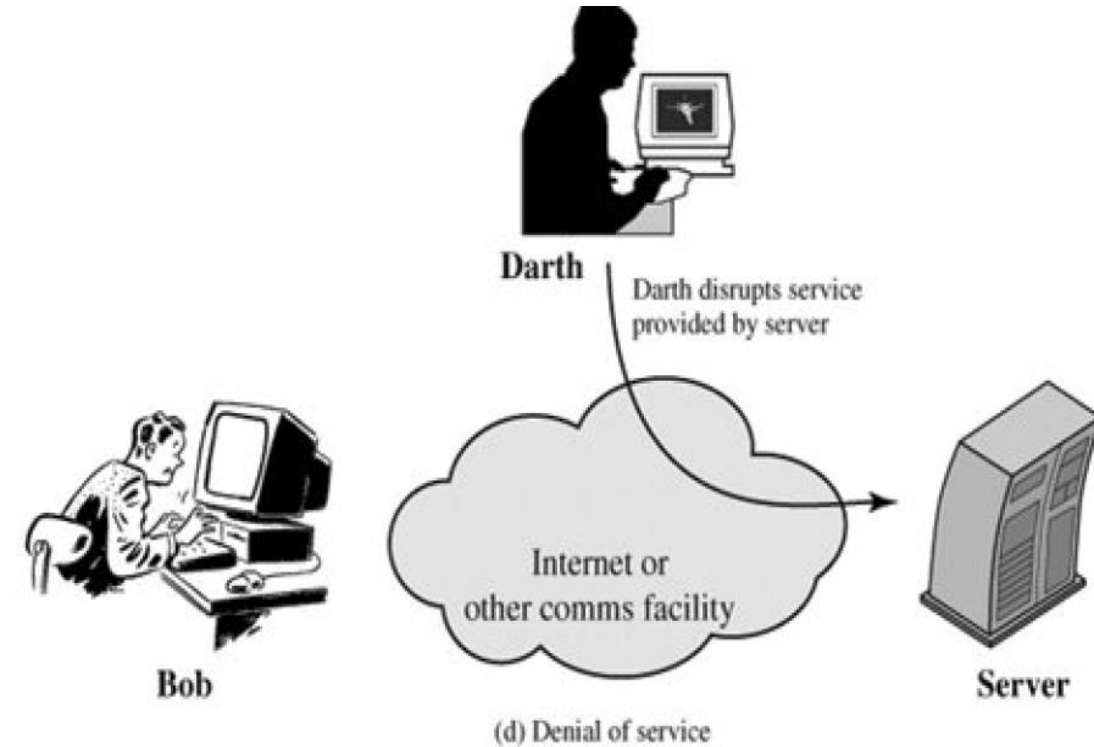
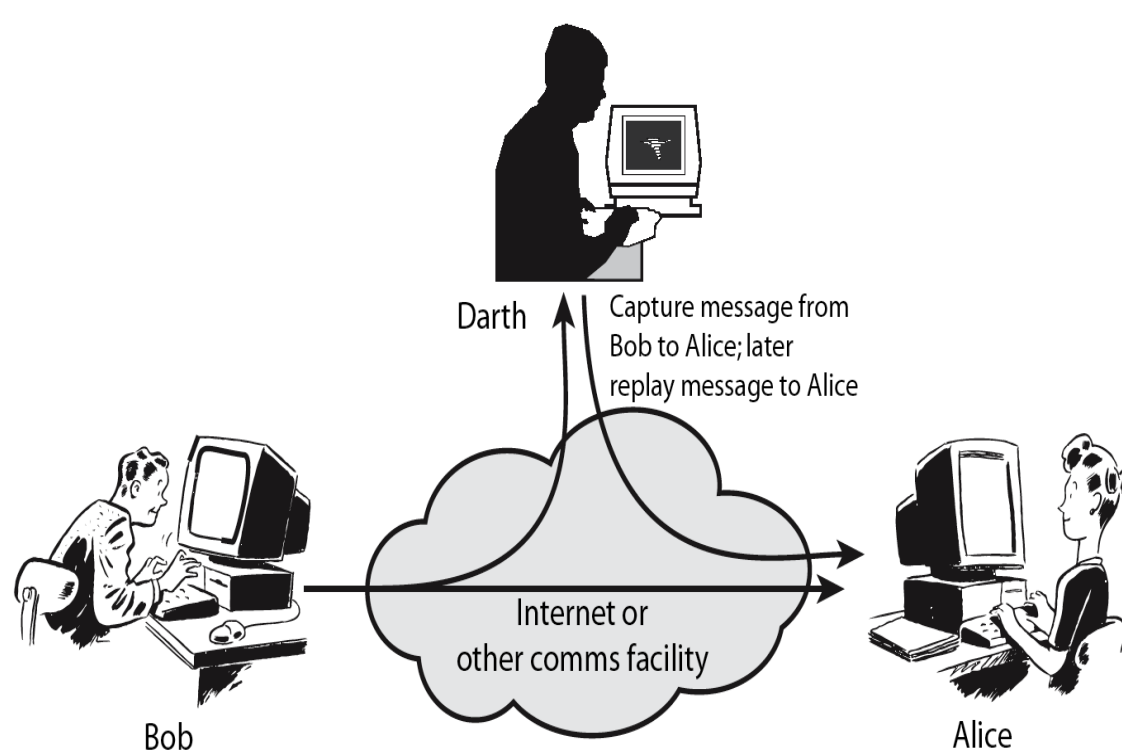


One entity pretends to be a different entity



Portion of the legitimate message is altered, delayed, recorded

Active Attacks

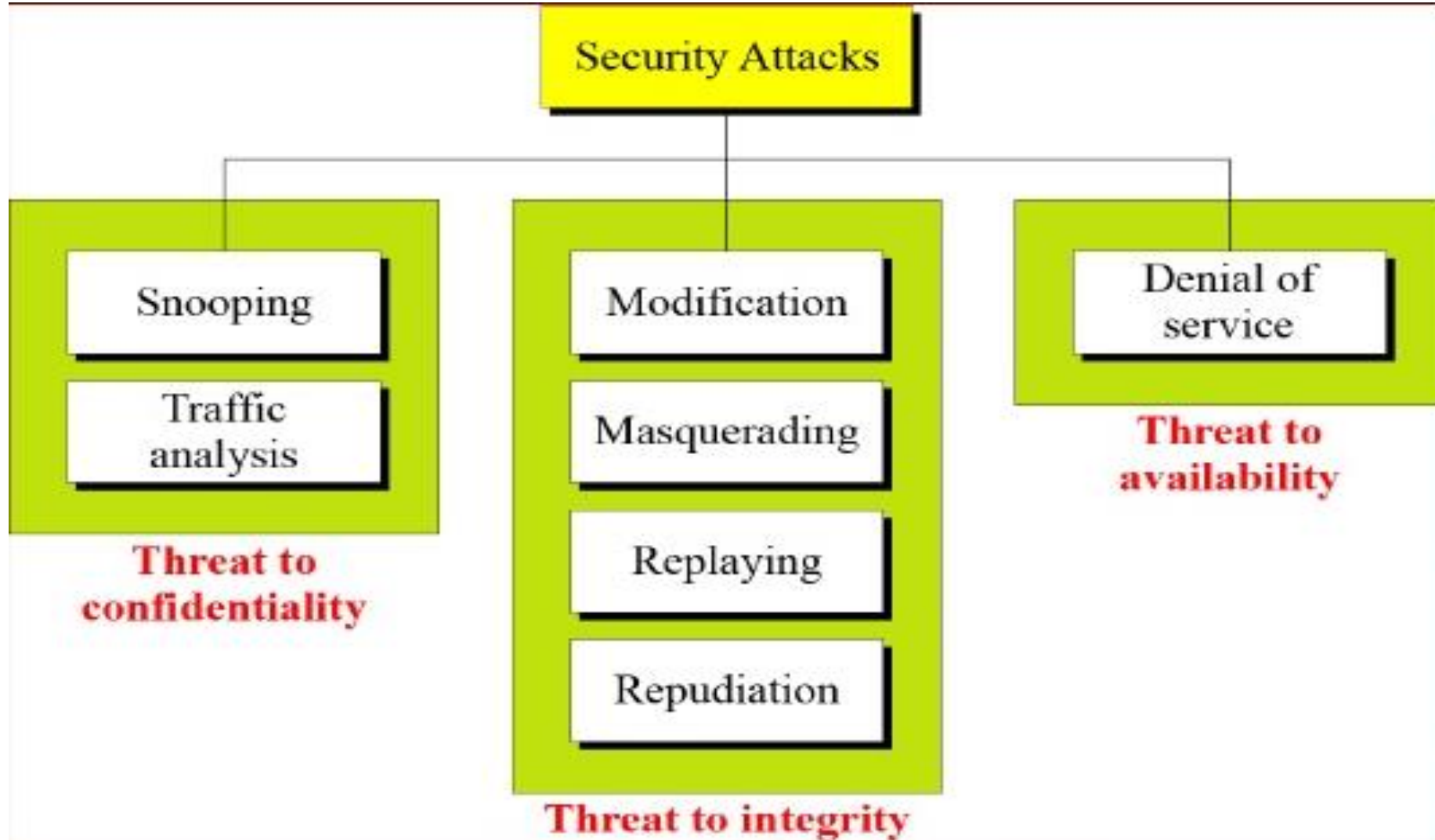


- Prevent the normal use or management of communication facility
1. Suppress all message directed to a particular destination
 2. Distrubtion of the entire network

Attack: Modification



Taxonomy of Attacks



Security Services

- X.800 defines it as: a service provided by a protocol layer of communicating open systems, which ensures adequate security of the systems or of data transfers
- RFC 2828 defines it as: a processing or communication service provided by a system to give a specific kind of protection to system resources
- X.800 defines it in 5 major categories

Security Services (X.800)

Authentication -
assurance that the
communicating entity is
the one claimed

Access Control -
prevention of the
unauthorized use of a
resource

Data Confidentiality –
protection of data from
unauthorized disclosure

Data Integrity -
assurance that data
received is as sent by an
authorized entity

Non-Repudiation -
protection against
denial by one of the
parties in a
communication

Availability- Protect a
system to ensure its
availability.

Security Mechanisms (X.800)

Specific security mechanisms:

- Encipherment, digital signatures, access controls, data integrity, authentication exchange, traffic padding, routing control, notarization

Pervasive security mechanisms:

- Trusted functionality, security labels, event detection, security audit trails, security recovery

SPECIFIC SECURITY MECHANISMS

May be incorporated into the appropriate protocol layer in order to provide some of the OSI security services.

Encipherment

The use of mathematical algorithms to transform data into a form that is not readily intelligible. The transformation and subsequent recovery of the data depend on an algorithm and zero or more encryption keys.

Digital Signature

Data appended to, or a cryptographic transformation of, a data unit that allows a recipient of the data unit to prove the source and integrity of the data unit and protect against forgery (e.g., by the recipient).

Access Control

A variety of mechanisms that enforce access rights to resources.

Data Integrity

A variety of mechanisms used to assure the integrity of a data unit or stream of data units.

Authentication Exchange

A mechanism intended to ensure the identity of an entity by means of information exchange.

Traffic Padding

The insertion of bits into gaps in a data stream to frustrate traffic analysis attempts.

Routing Control

Enables selection of particular physically secure routes for certain data and allows routing changes, especially when a breach of security is suspected.

Notarization

The use of a trusted third party to assure certain properties of a data exchange.

PERVASIVE SECURITY MECHANISMS

Mechanisms that are not specific to any particular OSI security service or protocol layer.

Trusted Functionality

That which is perceived to be correct with respect to some criteria (e.g., as established by a security policy).

Security Label

The marking bound to a resource (which may be a data unit) that names or designates the security attributes of that resource.

Event Detection

Detection of security-relevant events.

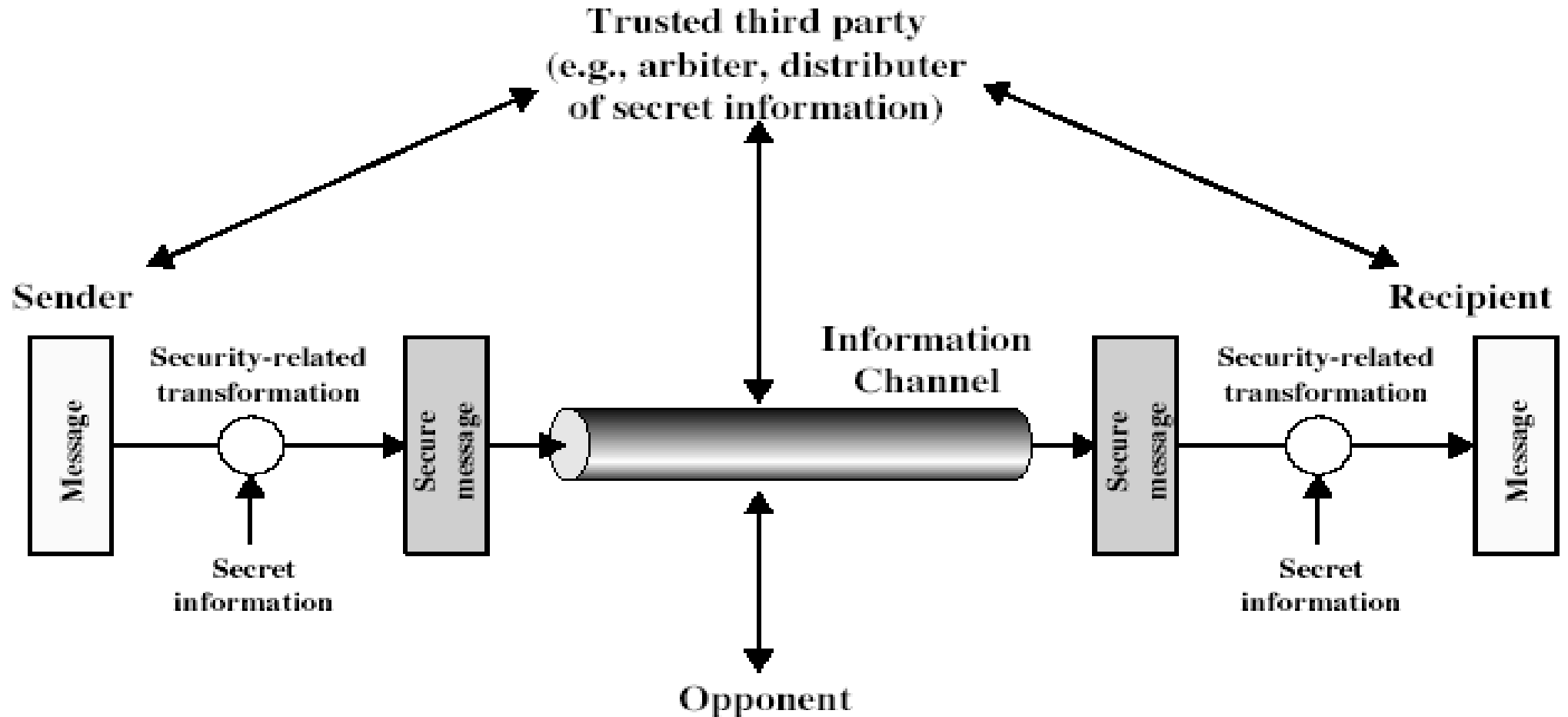
Security Audit Trail

Data collected and potentially used to facilitate a security audit, which is an independent review and examination of system records and activities.

Security Recovery

Deals with requests from mechanisms, such as event handling and management functions, and takes recovery actions.

Model for Network Security

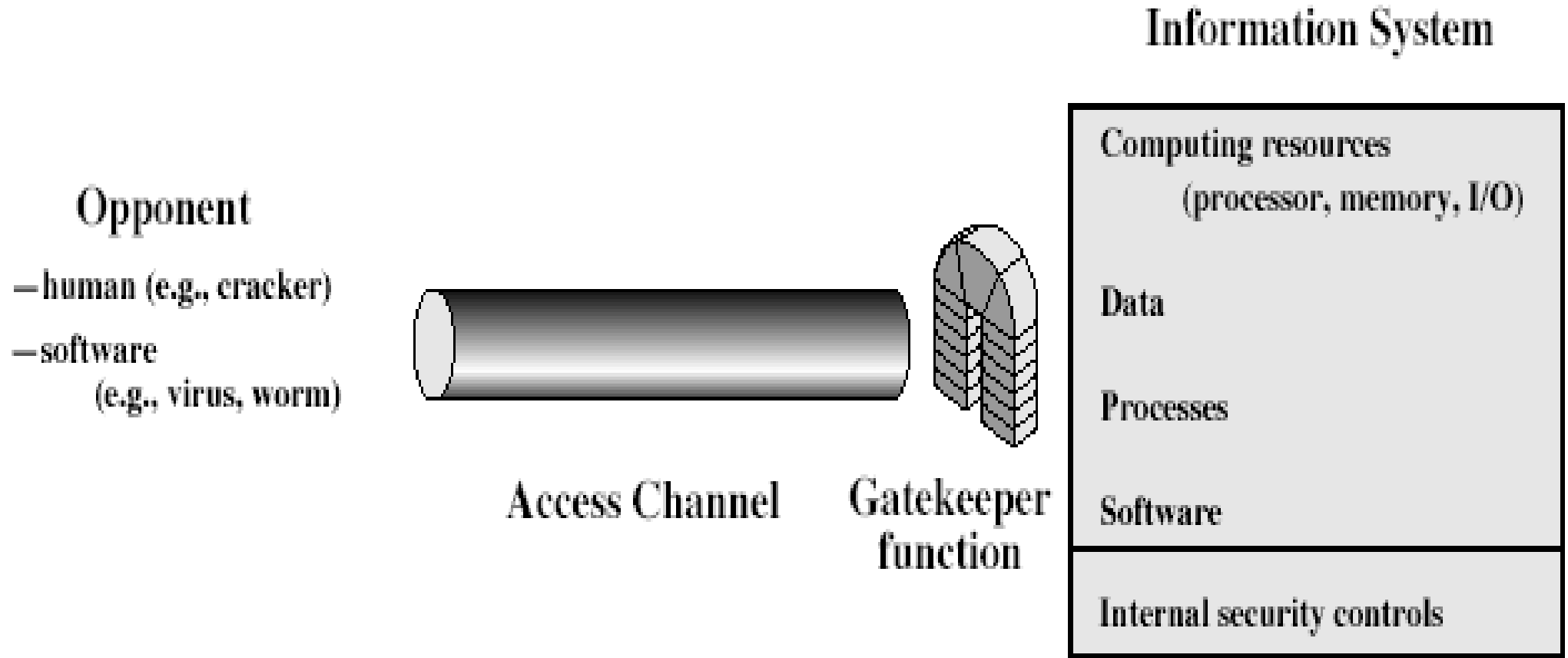


Model for Network Security

Using this model requires us to:

- Design a suitable algorithm for the security transformation
- Generate the secret information (keys) used by the algorithm
- Develop methods to distribute and share the secret information
- Specify a protocol enabling the principals to use the transformation and secret information for a security service

Model for Network Access Security



Model for Network Access Security

- Using this model requires us to:
 - Select appropriate gatekeeper functions to identify users
 - Implement security controls to ensure only authorised users access designated information or resources
- Trusted computer systems can be used to implement this model

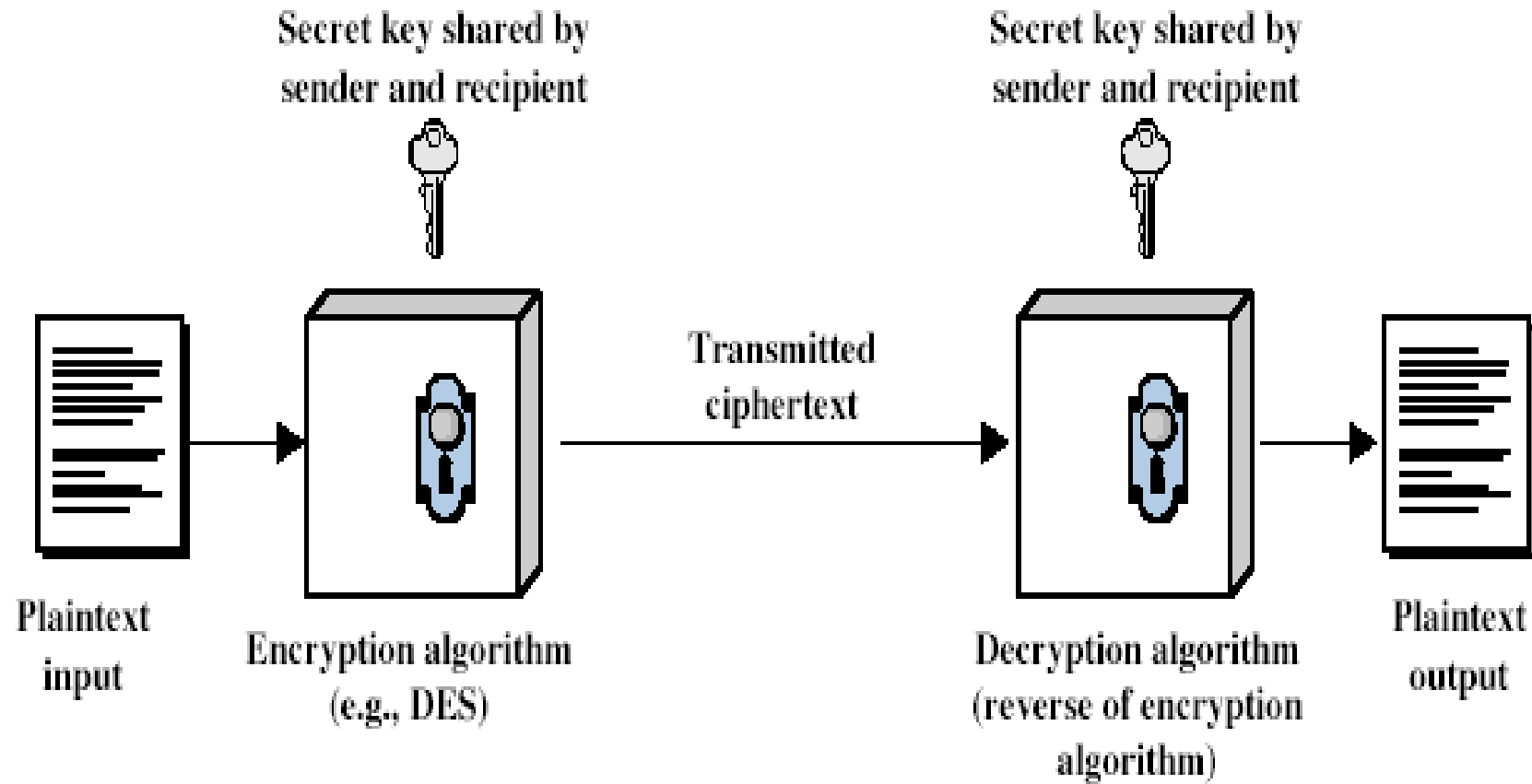
Symmetric Encryption

- Conventional / private-key / single-key
- Sender and recipient share a common key
- All classical encryption algorithms are private-key
- Was only type prior to invention of public-key in 1970's and by far most widely used

Some Basic Terminology

- **plaintext** - original message
- **ciphertext** - coded message
- **cipher** - algorithm for transforming plaintext to ciphertext
- **key** - info used in cipher known only to sender/receiver
- **encipher (encrypt)** - converting plaintext to ciphertext
- **decipher (decrypt)** - recovering ciphertext from plaintext
- **cryptography** - study of encryption principles/methods
- **cryptanalysis (codebreaking)** - study of principles/ methods of deciphering ciphertext *without* knowing key
- **cryptology** - field of both cryptography and cryptanalysis

Symmetric Cipher Model



Requirements

- two requirements for secure use of symmetric encryption:
 - a strong encryption algorithm
 - a secret key known only to sender / receiver
- mathematically have:
$$Y = E_K(X)$$
$$X = D_K(Y)$$
- assume encryption algorithm is known
- implies a secure channel to distribute key

Cryptography

Characterize cryptographic system by:

- Type of encryption operations used
 - Substitution: Each element in the plaintext (bit, letter, group of bits or letters) is mapped into another element.
 - Transposition: Elements in the plaintext are rearranged
 - Product: Involve multiple stages of substitutions and transpositions
- Number of keys used
 - Single-key or private : If both sender and receiver use the same key
 - Two-key or public : If the sender and receiver use different keys
- Way in which plaintext is processed
 - Block : processes the input one block of elements at a time, producing an output block for each input block
 - Stream : processes the input elements continuously, producing output one element at a time, as it goes along

Cryptanalysis

- Objective to **recover key** not just message
- General approaches:
 - **Cryptanalytic attack**
Rely on nature of algorithm plus perhaps some knowledge of the general characteristics of the plaintext
 - **Brute-force attack**
Try every possible key on a piece of ciphertext until an intelligible translation into plaintext is obtained.

Brute Force Search

- Always possible to simply try every key
- Most basic attack, proportional to key size
- Assume either know / recognise plaintext

Table 2.2. Average Time Required for Exhaustive Key Search

Key size (bits)	Number of alternative keys		Time required at 1 decryption/ <i>ms</i>		Time required at 10^6 decryption/ <i>ms</i>
32	2^{32}	$= 4.3 \times 10^9$	2^{31} ms	$= 35.8 \text{ minutes}$	2.15 milliseconds
56	2^{56}	$= 7.2 \times 10^{16}$	2^{55} ms	$= 1142 \text{ years}$	10.01 hours
128	2^{128}	$= 3.4 \times 10^{38}$	2^{127} ms	$= 5.4 \times 10^{24} \text{ years}$	$5.4 \times 10^{18} \text{ years}$
168	2^{168}	$= 3.7 \times 10^{50}$	2^{167} ms	$= 5.9 \times 10^{36} \text{ years}$	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26!$	$= 4 \times 10^{26}$	$2 \times 10^{26} \text{ ms}$	$= 6.4 \times 10^{12} \text{ years}$	$6.4 \times 10^6 \text{ years}$

More Definitions

- **Unconditional security**

- No matter how much **computer power or time** is available, the cipher cannot be broken since the ciphertext provides insufficient information to uniquely determine the corresponding plaintext

- **Computational security**

- The **cost of breaking the cipher exceeds the value** of the encrypted information
- The **time required to break the cipher exceeds** the useful lifetime of the information
- Given **limited computing resources** (eg time needed for calculations is greater than age of universe), the cipher cannot be broken

Classical Encryption Techniques

Basic Building blocks:

1. Substitution cipher
2. Transposition cipher
3. Product cipher

1. Substitution Ciphers

- where **letters of plaintext are replaced by other letters** or by numbers or symbols
- or **if plaintext is viewed as a sequence of bits, then substitution involves replacing plaintext bit patterns** with ciphertext bit patterns

Shift Cipher

- $Z_{26} = \{0, 1, 2, \dots, 24, 25\}$
- $\mathcal{P} = \mathcal{C} = \mathcal{K} = Z_{26}$
- For $k \in \mathcal{K}$,
 - $e_k(x) = (x + k) \bmod 26$ for $x \in \mathcal{P}$
 - $d_k(y) = (y - k) \bmod 26$ for $y \in \mathcal{C}$
- Caesar Cipher is a particular case (for $k = 3$)

Caesar Cipher

- Earliest known substitution cipher , by Julius Caesar
- First attested use in military affairs
- Replaces each letter by 3rd letter on
- Example:
 - meet me after the toga party
 - PHHW PH DIWHU WKH WRJD SDUWB

Encryption

$$c = E(p) = (p + k) \bmod 26$$

Decryption

$$p = D(c) = (c - k) \bmod 26$$

plain: a b c d e f g h i j k l m n o p q r s t u v w x y z

cipher: D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

Numerical
equivalent to
each letter

Cryptanalysis of Caesar Cipher

- Only have 26 possible ciphers
 - A maps to A,B,..Z
- Could **simply try each in turn**
- A **brute force search**
- Given ciphertext, just try all shifts of letters
- Do need to recognize when have plaintext
- **eg. break ciphertext "GCUA VQ DTGCM"**

KEY

1	oggv	og	chvgt	vjg	vqic	rctva
2	nffu	nf	bgufs	uif	uphb	qbsuz
3	meet	me	after	the	toga	party
4	ldds	ld	zesdq	sgd	snfz	ozqsx
5	kccr	kc	ydrpc	rfe	rmey	nyprw
6	jbbq	jb	xcqbo	geb	qldx	mxoqv
7	iaap	ia	wbpan	pda	pkcw	lwnpu
8	hzzo	hz	vaozm	ocz	ojbv	kvmot
9	gyyn	gy	uznyl	nby	niau	julns
10	fxxm	fx	tymxk	max	mhzt	itkmr
11	ewwl	ew	sxlwj	lzw	lgys	hsjllq
12	dvvk	dv	rwkvi	kyv	kfxr	grikp
13	cuuj	cu	qvjuh	jxu	jewq	fqhjo
14	btti	bt	puitg	iwt	idvp	epgin
15	assh	as	othsf	hvs	hcuo	dofhm
16	zrrg	zr	nsgre	gur	gbtn	cnegl
17	yqqf	yq	mrfqd	ftq	fasm	bmdfk
18	xppe	xp	lqepc	esp	ezrl	alcej
19	wood	wo	kpdob	dro	dyqk	zkbdi
20	vnnc	vn	jocna	cqn	cxpj	yjach
21	ummb	um	inbmz	bpm	bwoi	xizbg
22	tlla	tl	hmaly	aol	avnh	whyaf
23	skkz	sk	glzkx	znk	zumg	vgxze
24	rjjy	rj	fkyjw	ymj	ytlf	ufwyd
25	qiix	qi	ejxiv	xli	xske	tevxc

Example tools

- <https://www.dcode.fr/caesar-cipher> (Caesar cipher)
- **eg. break ciphertext “xli ryqfiv sj oicw”**

Monoalphabetic Cipher

- Rather than just shifting the alphabet, could shuffle (jumble) the letters arbitrarily
- **Each plaintext letter maps to a different random ciphertext letter**
- Hence key is 26 letters long

Plain: abcdefghijklmnopqrstuvwxyz

Key : DKVQFIBJWPESCXHTMYAUOLRGZN

Eg: if we wish to replace letters

Plaintext: ifwewishtoreplaceletters

Ciphertext: WIRFRWAJUHYFTSDVFSFUUFYA

LEARN
IN
TECHNOLOGY
AND
INNOVATION

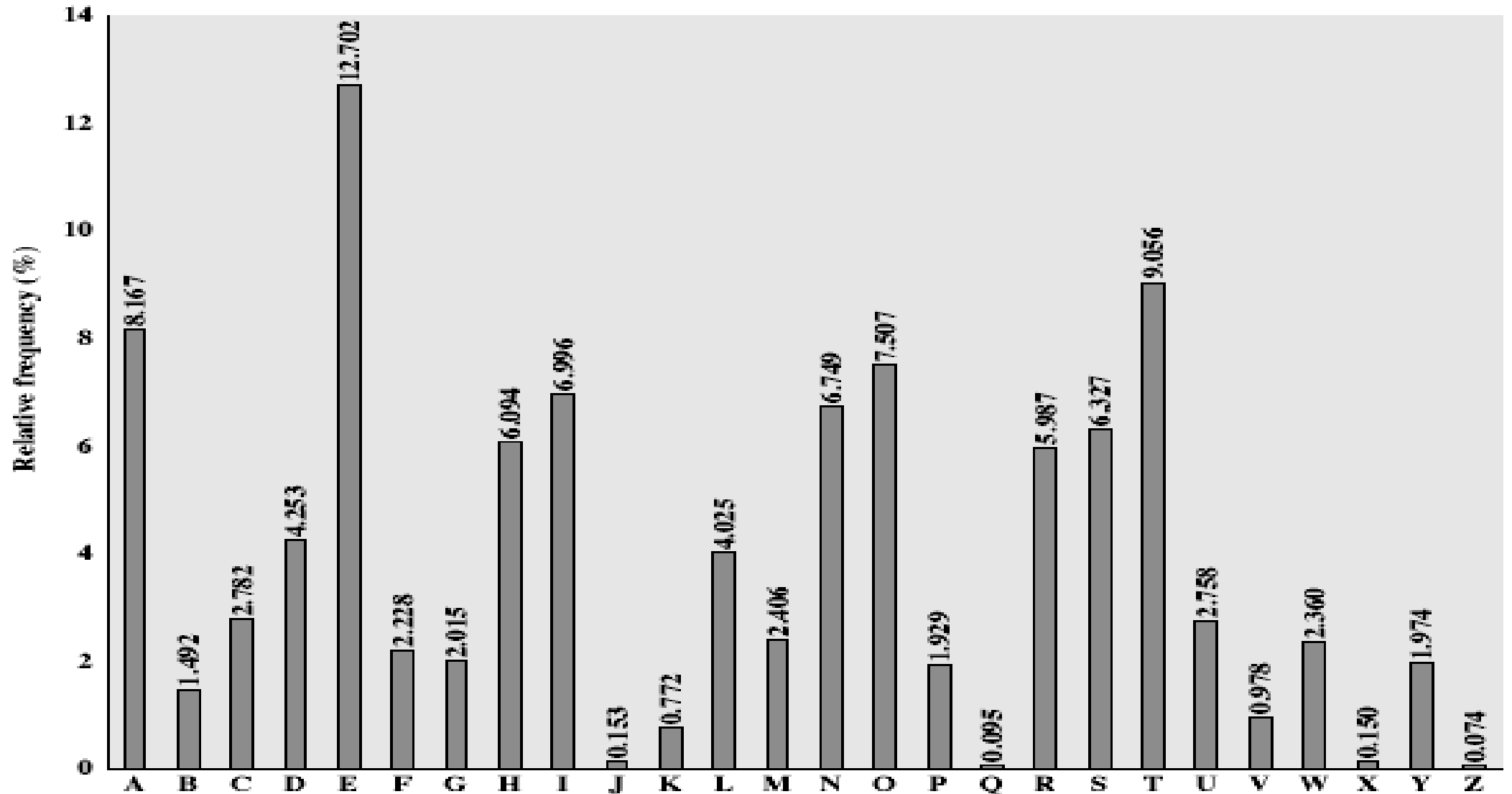
Monoalphabetic Cipher Security

- now have a total of 26! keys
- with so many keys, might think is secure
- but would be **!!!WRONG!!!**
- problem is language characteristics

Language Redundancy and Cryptanalysis

- Human languages are **redundant**
 - Eg : "th lrd s m shphrd shll nt wnt"
- Letters are not equally commonly used
- In English **E is by far the most common letter** followed by **T,R,N,I,O,A,S**
- Letters like **Z,J,K,Q,X** are fairly rare
- Have tables of single, double & triple letter frequencies for various languages
 - Two letter(digram): th, er, on,an,re,hi,in,id ...
 - Three letter(trigram): the, and, tha, ent....

English Letter Frequencies



Example Cryptanalysis

- Given ciphertext:

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
SFPAPPDTSVPQUZWYMXUZUHSXEPYEPDPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ

- count relative letter frequencies (see text)
- guess P & Z are **e** and **t**
- guess ZW is **th** and hence ZWP is **the**
- proceeding with trial and error :

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
t a e e te a that e e a a
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
e t ta t ha e ee a e th t a
EPYEPDPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
e e e tat e the t

Relative frequency of ciphertext in %

P 13.33	H 5.83	F 3.33	B 1.67	C 0.00
Z 11.67	D 5.00	W 3.33	G 1.67	K 0.00
S 8.33	E 5.00	Q 2.50	Y 1.67	L 0.00
U 8.33	V 4.17	T 2.50	I 0.83	N 0.00
O 7.50	X 4.17	A 1.67	J 0.83	R 0.00
M 6.67				

Finally :

it was disclosed yesterday that several informal but direct contacts have been made with political representatives of the viet cong in moscow

Playfair Cipher

- Not even the large number of keys in a monoalphabetic cipher provides security
- One approach to improving security was to encrypt multiple letters
- The **Playfair Cipher** is an example
- Invented by **Charles Wheatstone in 1854**, but named after his friend Baron Playfair

Playfair Key Matrix

- A 5X5 matrix of letters based on a keyword
- Fill in letters of keyword
- Fill rest of matrix with other letters
- Eg. using the keyword MONARCHY

M	O	N	A	R
C	H	Y	b	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Encrypting and Decrypting

plaintext is encrypted two letters(digram) at a time

1. if a pair is a repeated letter, insert filler like 'X'
2. if both letters fall in the same row, replace each with letter to right (wrapping back to start from end)
3. if both letters fall in the same column, replace each with the letter below it (again wrapping to top from bottom)
4. otherwise each letter is replaced by the letter in the same row and in the column of the other letter of the pair

Example

- Plaintext is encrypted two letters at a time.
 - if a pair is a repeated letter, insert a filler like 'X', eg. "balloon" encrypts as "ba lx lo on"
 - if both letters fall in the same row, replace each with letter to right (wrapping back to start from end), eg. "ar" encrypts as "RM"
 - if both letters fall in the same column, replace each with the letter below it (again wrapping to top from bottom), eg. "mu" encrypts to "CM"
 - otherwise each letter is replaced by the one in its row in the column of the other letter of the pair, eg. "hs" encrypts to "BP", and "ea" to "IM" or "JM" (as desired)
- Decryption works exactly in reverse. **E.g hmclprhmsuulif**

M	O	N	A	R
C	H	Y	b	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Example

Plain Text: "in st ru me nt sz"

Encrypted Text: ga tl mz cl rq tx

Encryption: in -> ga, st-> tl, ru -> mz, me -> cl, nt -> rq, sz -> tx

in:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

st:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

ru:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

me:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

nt:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

sz:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

Playfair Cipher

- Use the keyword CHARLES (Charles Wheatstone invented the cipher).
- Draw up a 5×5 matrix with the keyword first, removing any repeating letters as follows:

c	h	a	r	l
e	s	b	d	f
g	i/j	k	m	n
o	p	q	t	u
v	w	x	y	z

- Plaintext: “meet me at the bridge”.
 - Split the sentence into digrams removing spaces, ‘x’ used to make even number of letters:
me et me at th eb ri dg ex
 - Repeating plaintext letters that are in the same pair are separated with a filler letter, such as x:
“balloon” would be treated as ba lx lo on

- Ciphertext therefore is :

“gd do gd rq pr sd hm em bv”

Rules for Decryption:

If both the letters are in the same column: Take the letter above each one (going back to the bottom if at the top).

If both the letters are in the same row: Take the letter to the left of each one (going back to the rightmost if at the leftmost position).

If neither of the above rules is true: Form a rectangle with the two letters and take the letters on the horizontal opposite corner of the rectangle.

Security of Playfair Cipher

- Security much improved over monoalphabetic since have $26 \times 26 = 676$ digrams
- Would need a 676 entry frequency table to analyse (verses 26 for a monoalphabetic) and correspondingly more ciphertext
- Was widely used for many years
 - eg. by US & British military in WW1
- It **can** be broken, given a few hundred letters of ciphertext since still has much of plaintext structure

Transposition Ciphers

- Classical **transposition** or **permutation** ciphers hide the message by rearranging the letter order without altering the actual letters used
- Can recognise these since have the same frequency distribution as the original text

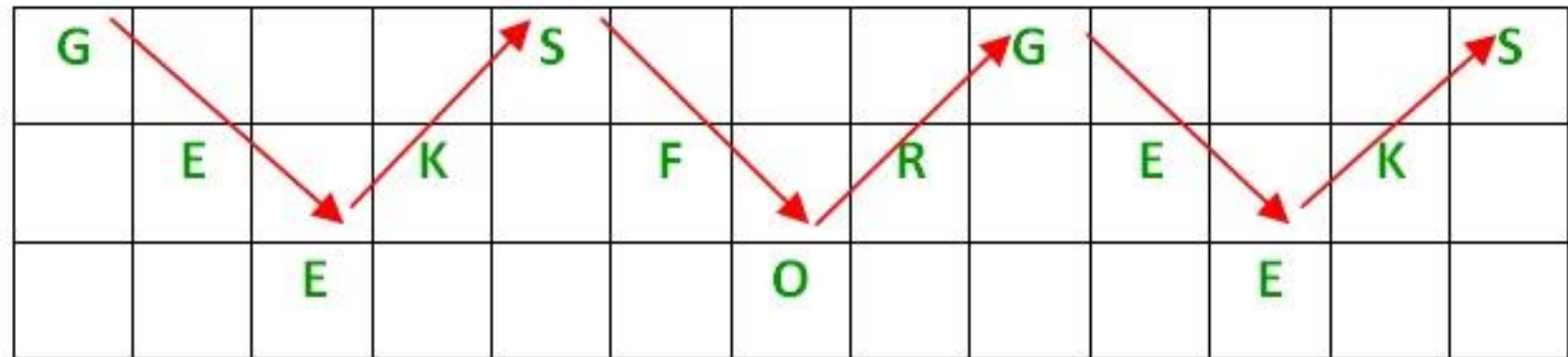
Rail Fence Cipher

- Write message letters out diagonally over a number of rows
- Then read off cipher row by row
- eg. write message out as:

m e m a t r h t g p r y
e t e f e t e o a a t

- giving ciphertext

MEMATRHTGPRYETEFETEOAAT



Decrypt

- Rail fence cipher
- “nosrewr euiytkct”
- “tonrpsoasiint”

LEARNED AUGMENTED

- | | | | | | | | |
|------------|----------|----------|----------|----------|----------|----------|----------|
| Key: | 4 | 3 | 1 | 2 | 5 | 6 | 7 |
| Plaintext: | a | t | t | a | c | k | p |
| | o | s | t | p | o | n | e |
| | d | u | n | t | i | l | t |
| | w | o | a | m | x | y | z |

Find out plaintext : aeinsnoillthcapcscngirtucyee

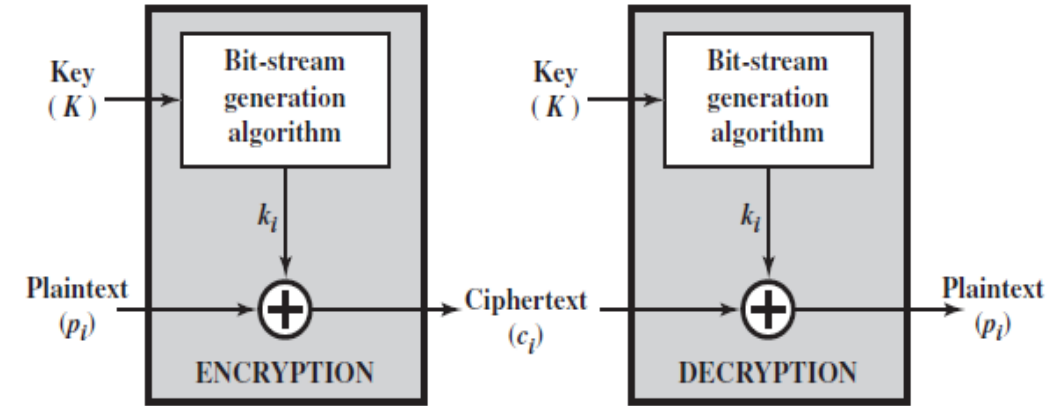
Product Ciphers

- Ciphers using substitutions or transpositions are not secure because of language characteristics
- Hence consider using several ciphers in succession to make harder, but:
 - two substitutions make a more complex substitution
 - two transpositions make more complex transposition
 - but a substitution followed by a transposition makes a new much harder cipher
- This is bridge from classical to modern ciphers

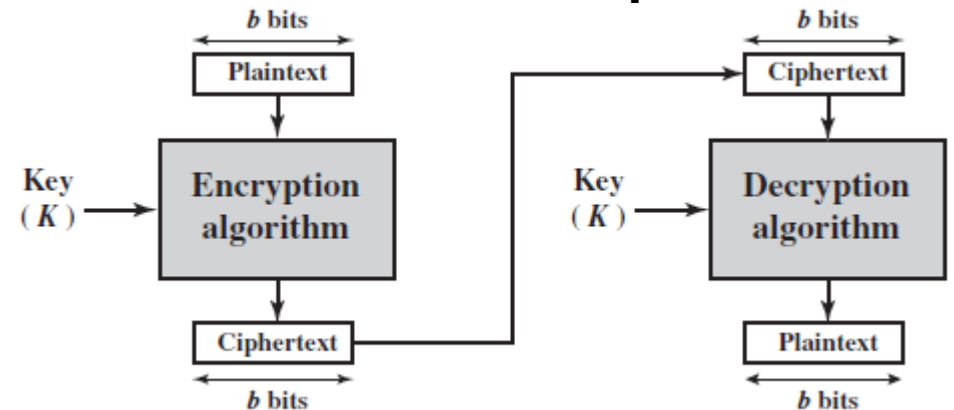
Block vs Stream Ciphers

- A **stream cipher** is one that encrypts a digital data stream **one bit or one byte at a time**.
 - Examples of classical stream ciphers are the auto keyed Vigenère cipher and the Vernam cipher.
- A **block cipher** is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length
- many current ciphers are block ciphers

stream cipher



block cipher



Block Cipher Principles

1. most **symmetric block ciphers** are based on a **Feistel Cipher Structure**
2. needed since must be able to **decrypt** ciphertext to recover messages efficiently
3. block ciphers look like an extremely large substitution
4. would need table of 2^{64} entries for a 64-bit block
5. using idea of a product cipher

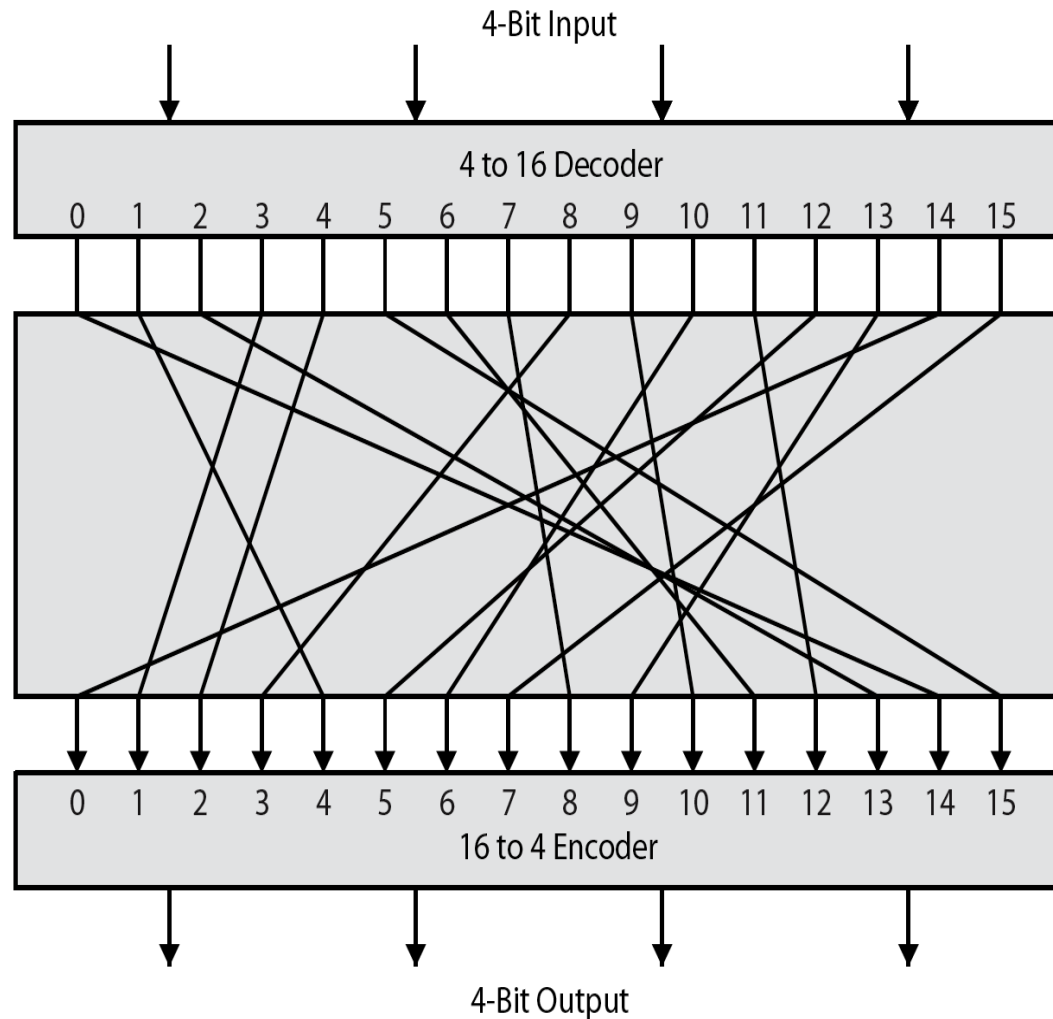
Reversible Mapping

Plaintext	Ciphertext
00	11
01	10
10	00
11	01

Irreversible Mapping

Plaintext	Ciphertext
00	11
01	10
10	01
11	01

Ideal Block Cipher



Reversible Mapping

Plaintext	Ciphertext
00	11
01	10
10	00
11	01

Irreversible Mapping

Plaintext	Ciphertext
00	11
01	10
10	01
11	01

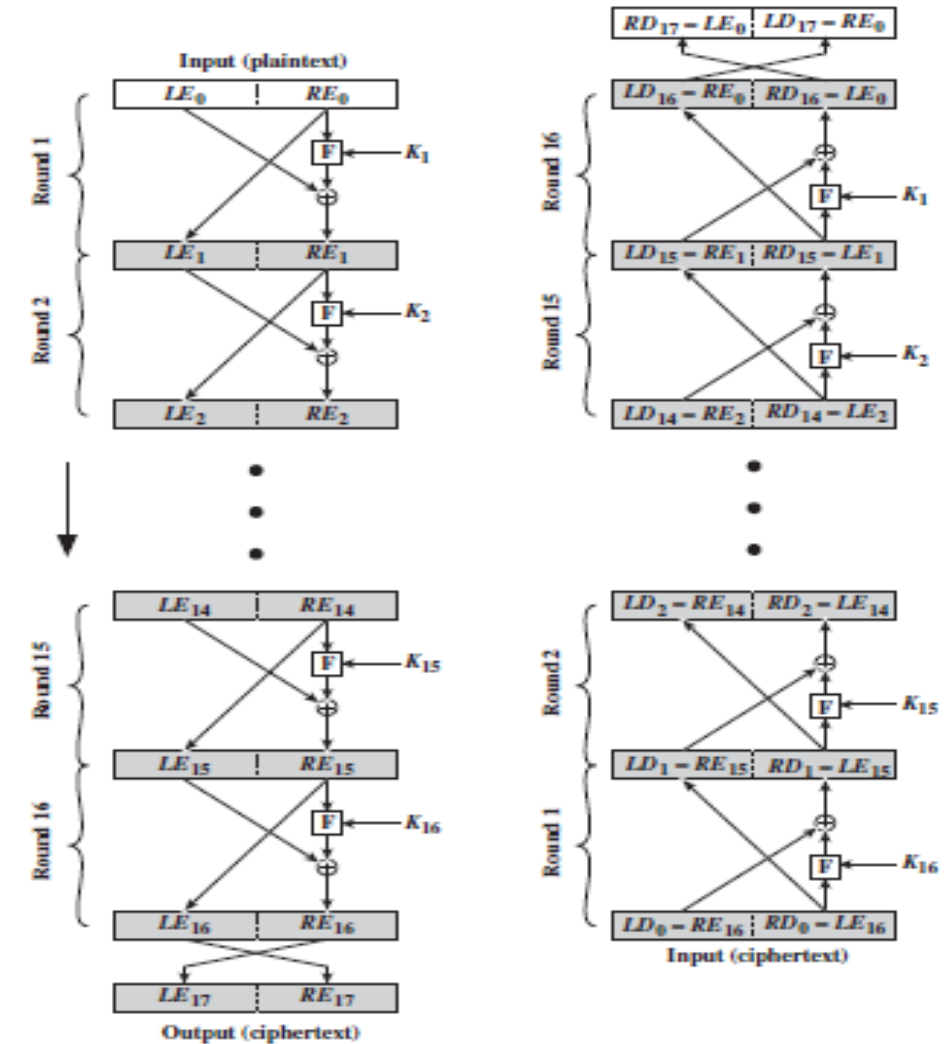
The Feistel Cipher

- **Substitution:** Each plaintext element or group of elements is uniquely replaced by a corresponding ciphertext element or group of elements.
- **Permutation:** A sequence of plaintext elements is replaced by a permutation of that sequence.
 - I.e. no elements are added or deleted or replaced in the sequence, rather the order in which the elements appear in the sequence is changed.
- Feistel's is a practical application of a proposal by **Claude Shannon**
- Claude Shannon introduced the terms *diffusion* and *confusion* were introduced to capture the **two basic building blocks for any cryptographic system**
 - **Diffusion**, the statistical structure of the plaintext is dissipated into long-range statistics of the ciphertext.
 - **confusion** makes relationship between ciphertext and key as complex as possible

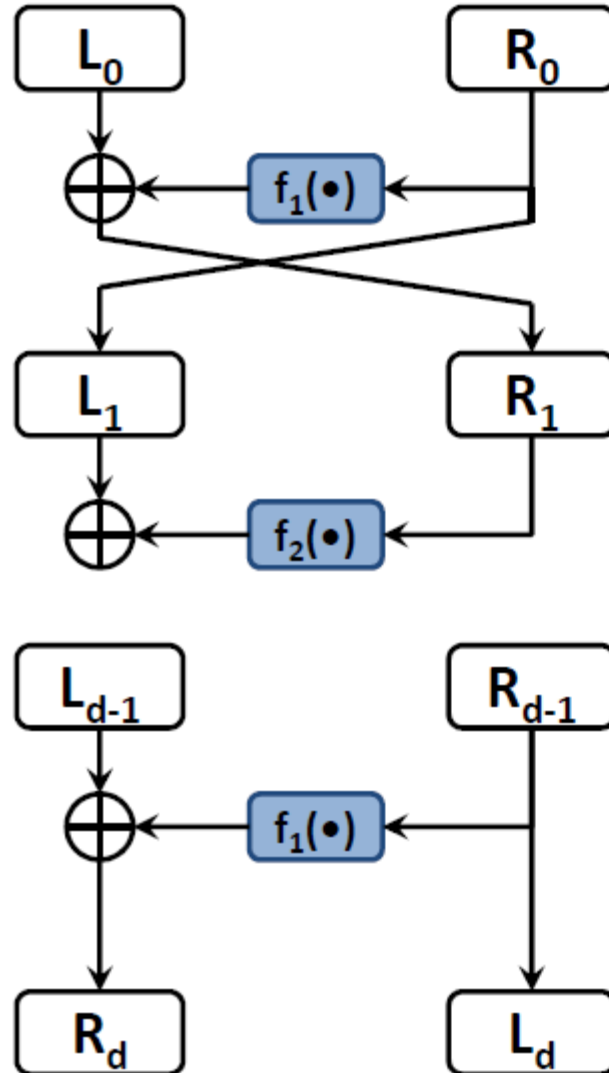
Feistel Cipher Structure

- Horst Feistel devised the **feistel cipher**
- based on concept of invertible product cipher
- The inputs to the encryption algorithm are a **plaintext block of length $2w$ bits and a key K** .
- The plaintext block is **divided into two halves**, L_0 and R_0 .
- The two halves of the data pass through **n rounds of processing** and then combine to produce the ciphertext block.
- Each round i has as inputs **L_{i-1} and R_{i-1}** derived from the previous round,
- A subkey K_i derived from the overall K .
- The subkeys K_i are different from K and from each other.

- Encryption :** $LE_{16} = RE_{15}$, $RE_{16} = LE_{15} + F(RE_{15}, K_{16})$
- On decryption side,**
 - $LD_1 = RD_0 = LE_{16} = RE_{15}$
 - $RD_1 = LD_0 + F(RD_0, K_{16})$
 - $= RE_{16} + F(RE_{15}, K_{16}) = [LE_{15} + F(RE_{15}, K_{16})] + F(RE_{15}, K_{16})$
 - $RD_1 = LE_{15}$



Feistel Network



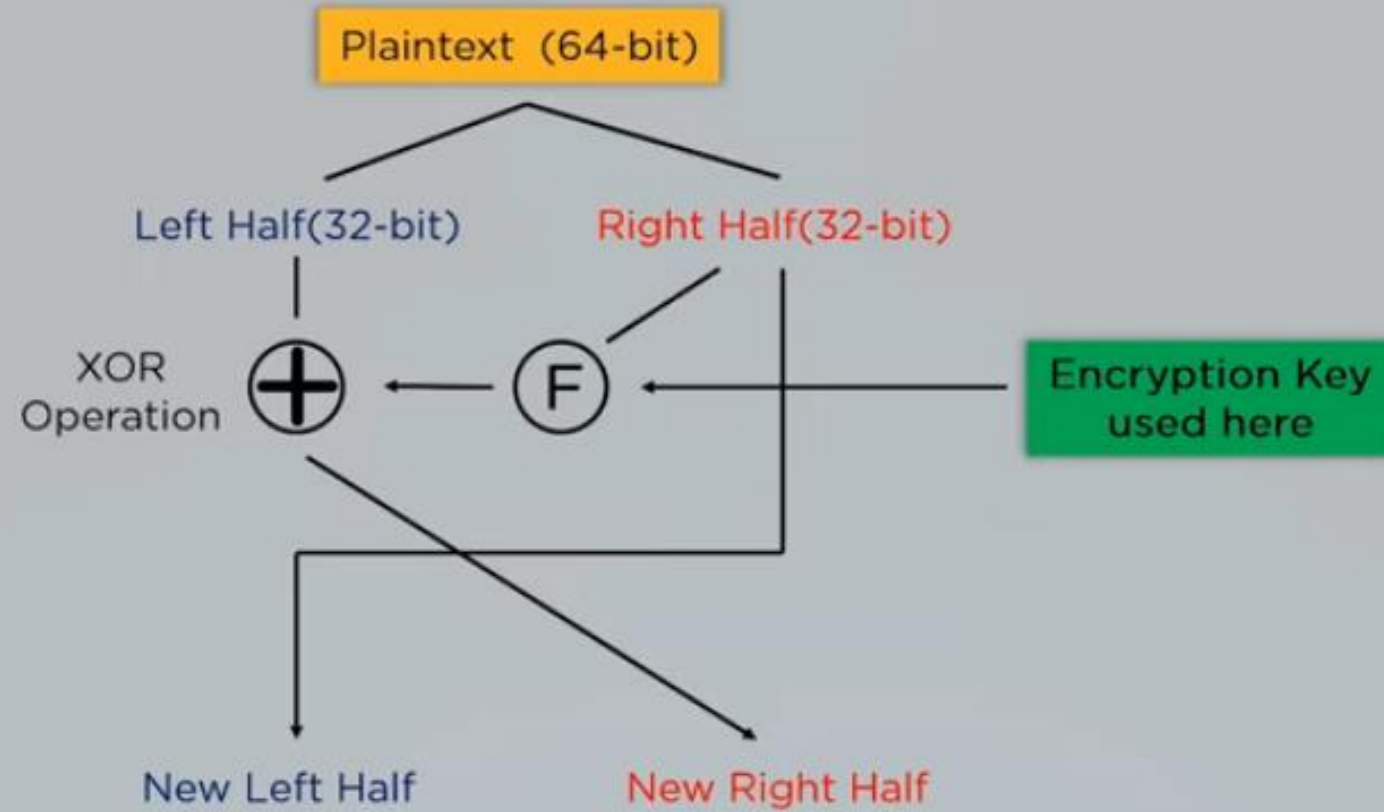
• Encryption:

- $L_1 = R_0$ $R_1 = L_0 \oplus f_1(R_0)$
- $L_2 = R_1$ $R_2 = L_1 \oplus f_2(R_1)$
- ...
- $L_d = R_{d-1}$ $R_d = L_{d-1} \oplus f_d(R_{d-1})$

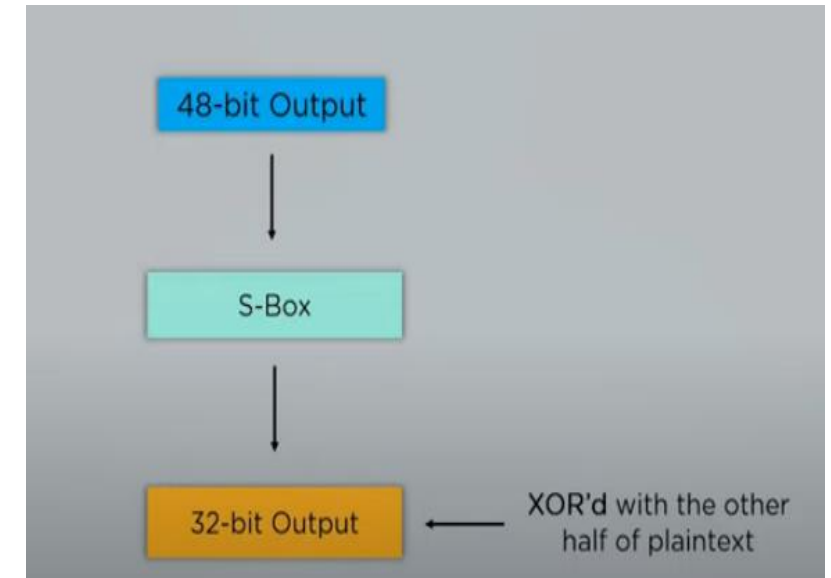
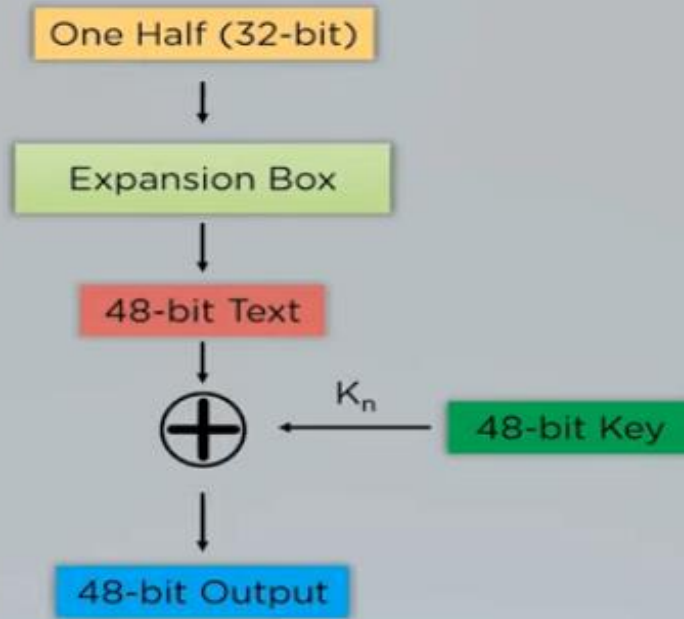
• Decryption:

- $R_{d-1} = L_d$ $L_{d-1} = R_d \oplus f_d(L_d)$
- ...
- $R_0 = L_1$; $L_0 = R_1 \oplus f_1(L_1)$

Feistel Ciphers



Round Function

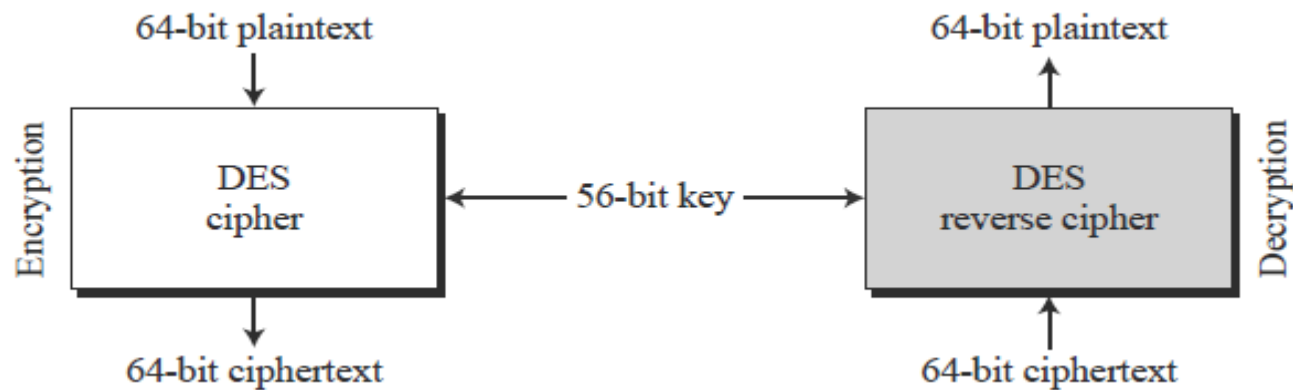


Feistel Cipher Design Elements

- block size
- key size
- number of rounds
- subkey generation algorithm
- round function F
- fast software en/decryption
- ease of analysis

Data Encryption Standard (DES)

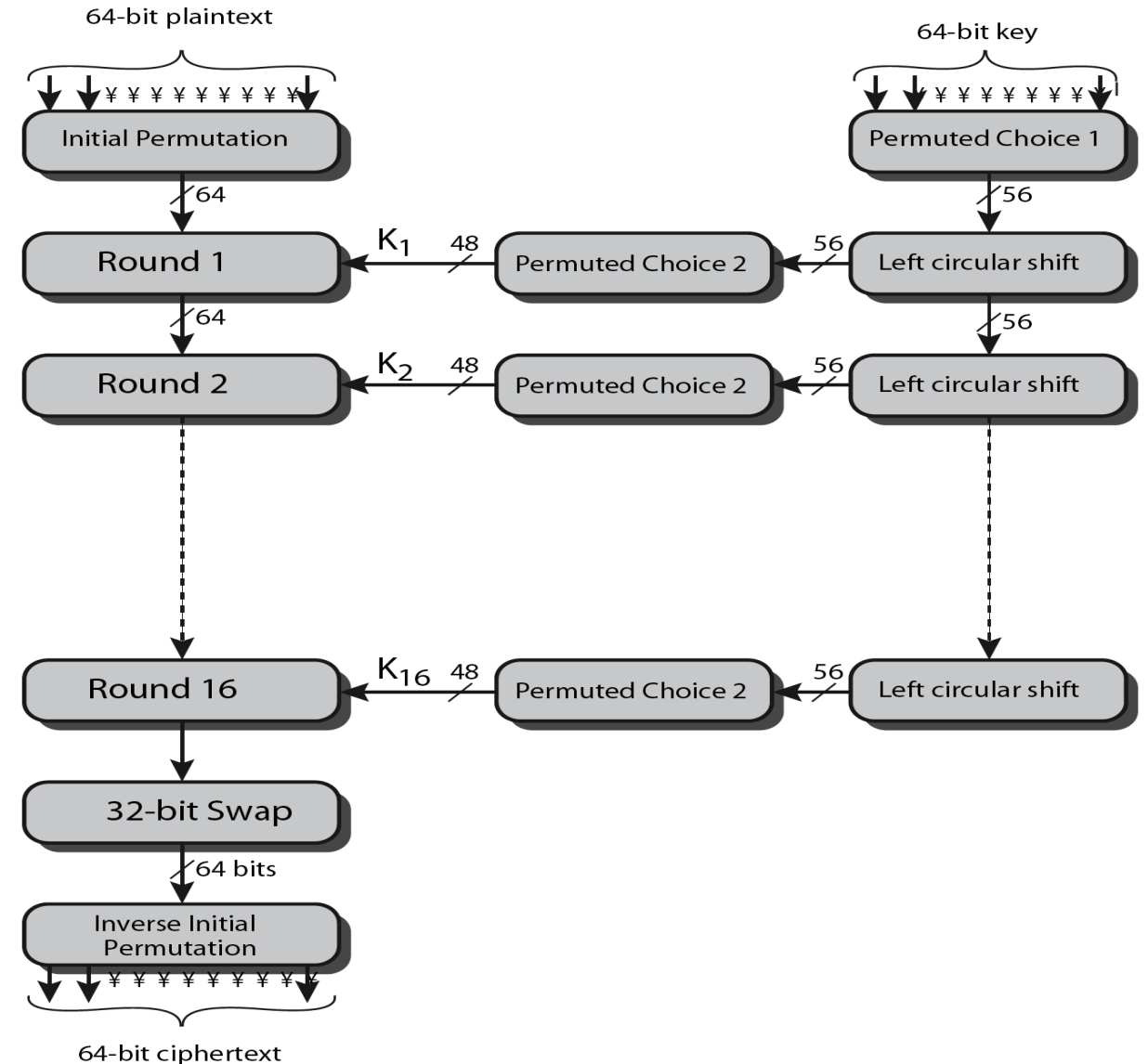
- most widely used block cipher in world
- adopted in 1977 by NBS (now NIST)
- encrypts **64-bit data using 56-bit key**
- has widespread use



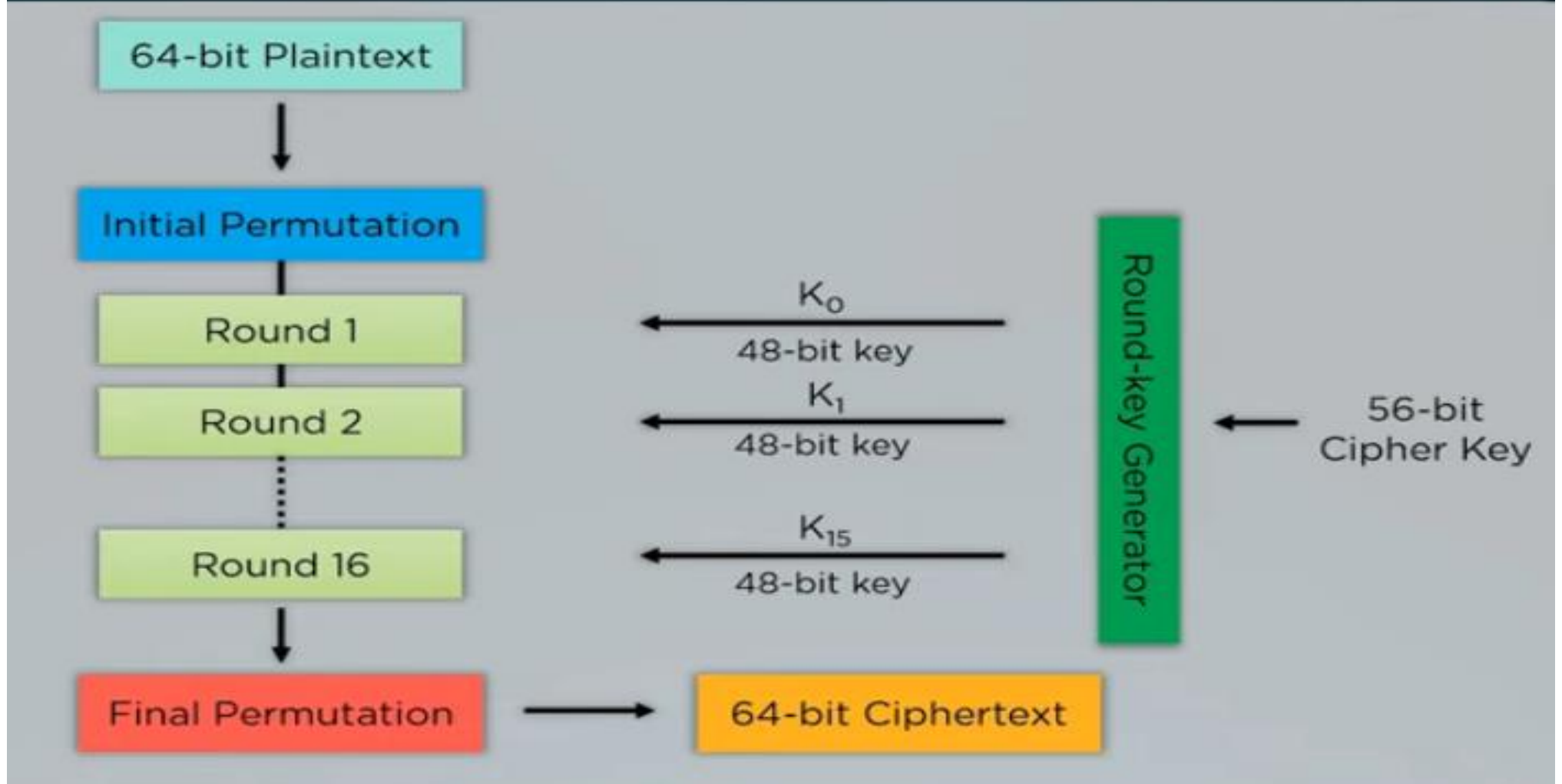
DES Encryption Overview

- **64-bit plaintext** passes through an initial permutation (IP) that rearranges the bits to produce the **permuted input**.
- **sixteen rounds** of the same function, which involves both permutation and substitution functions.
- The output of the last (sixteenth) round consists of 64 bits that are a function of the input plaintext and the key.
- **The left and right halves** of the output are swapped to produce the **preoutput**.
- Finally, the **preoutput** is passed through a permutation [IP⁻¹] that is the inverse of the initial permutation function, **to produce the 64-bit ciphertext**.

15-Apr-26



How DES Works – Key Generation

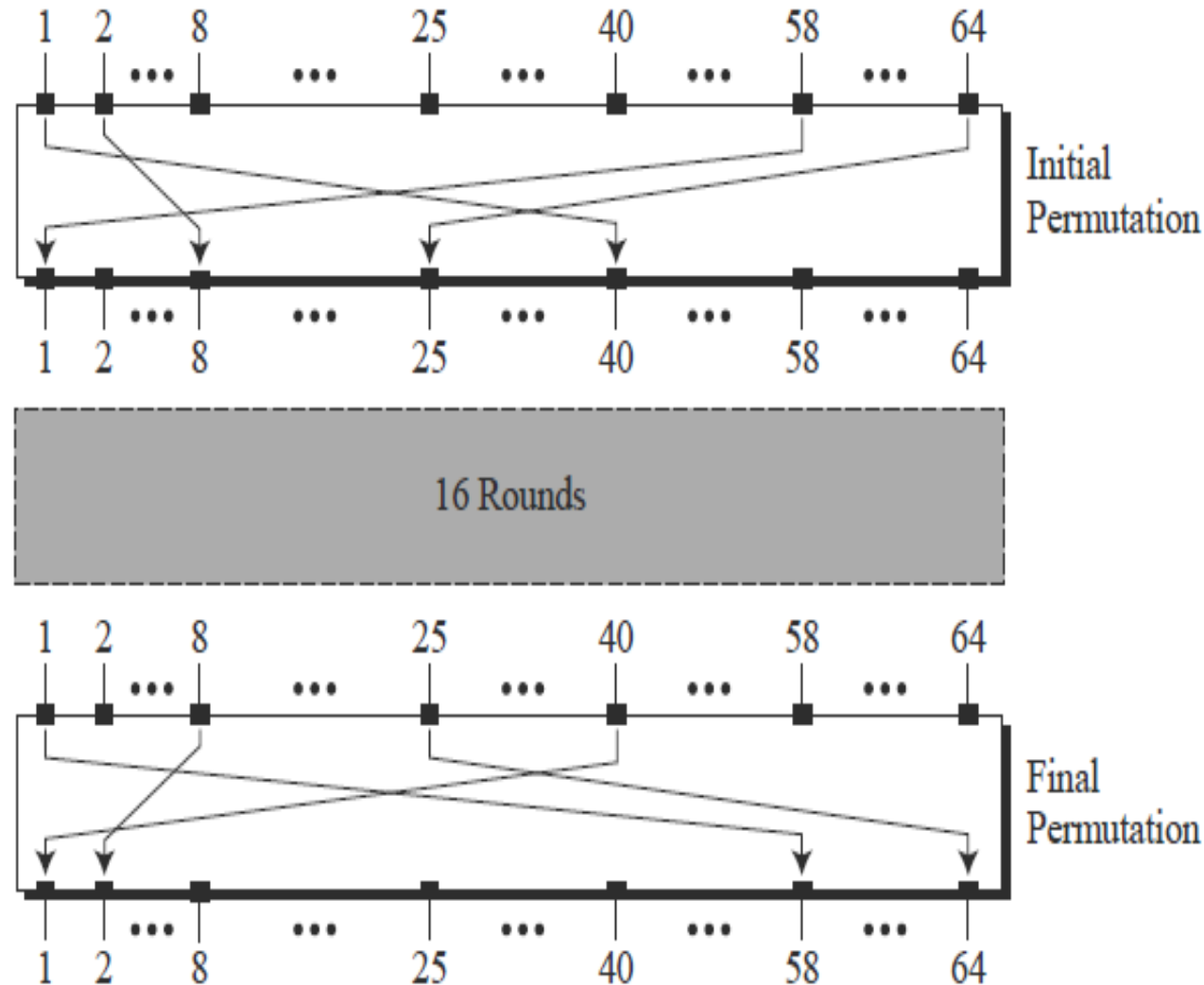


Initial Permutation IP

- first step of the data computation
- IP reorders the input data bits
- even bits to LH half, odd bits to RH half
- quite regular in structure (easy in h/w)

Initial and Final Permutations

P-Box



Initial Permutation	Final Permutation
58 50 42 34 26 18 10 02	40 08 48 16 56 24 64 32
60 52 44 36 28 20 12 04	39 07 47 15 55 23 63 31
62 54 46 38 30 22 14 06	38 06 46 14 54 22 62 30
64 56 48 40 32 24 16 08	37 05 45 13 53 21 61 29
57 49 41 33 25 17 09 01	36 04 44 12 52 20 60 28
59 51 43 35 27 19 11 03	35 03 43 11 51 19 59 27
61 53 45 37 29 21 13 05	34 02 42 10 50 18 58 26
63 55 47 39 31 23 15 07	33 01 41 09 49 17 57 25

$$Y = IP^{-1}(X) = IP^{-1}(IP(M))$$

The meaning is as follows:

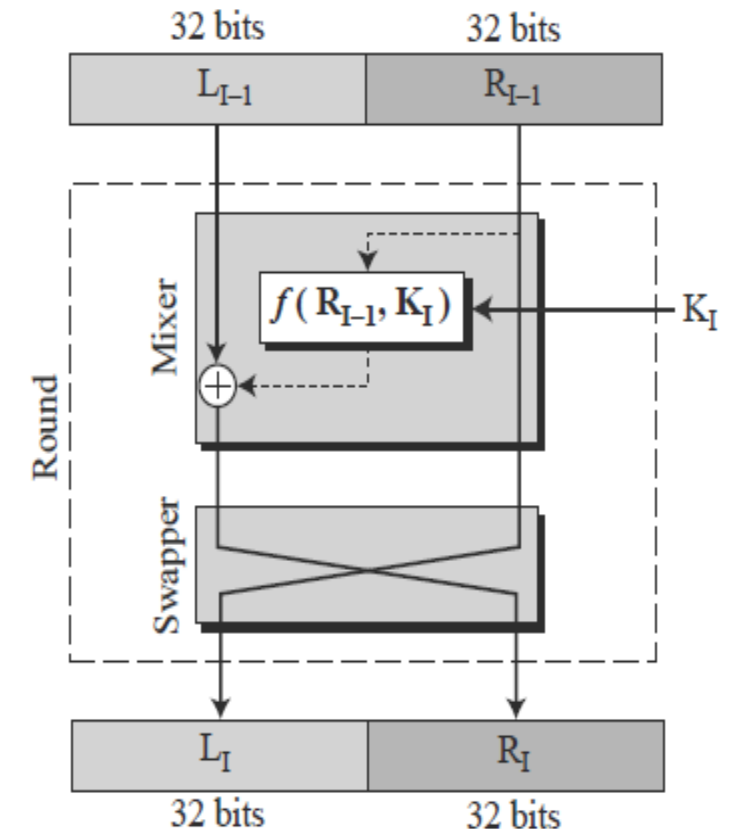
- the first bit of the output is taken from the 58th bit of the input; the second bit from the 50th bit, and so on, with the last bit of the output taken from the 7th bit of the input.

DES Round Structure

- uses two 32-bit L & R halves
- as for any Feistel cipher can describe as:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$
- F takes 32-bit R half and 48-bit subkey:
 - expands R to 48-bits using perm E
 - adds to subkey using XOR
 - passes through 8 S-boxes to get 32-bit result
 - finally permutes using 32-bit perm P



DES Round Structure

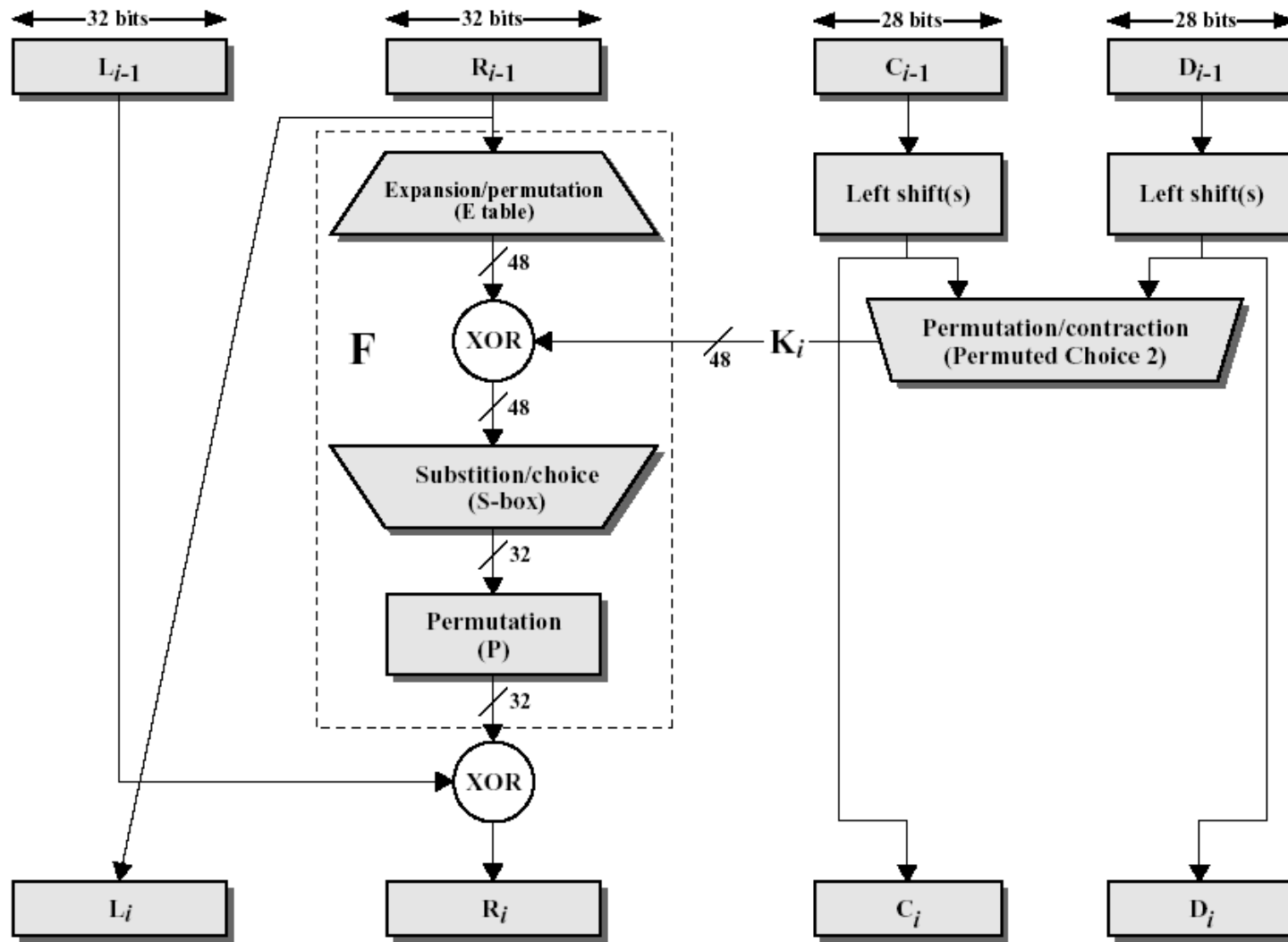
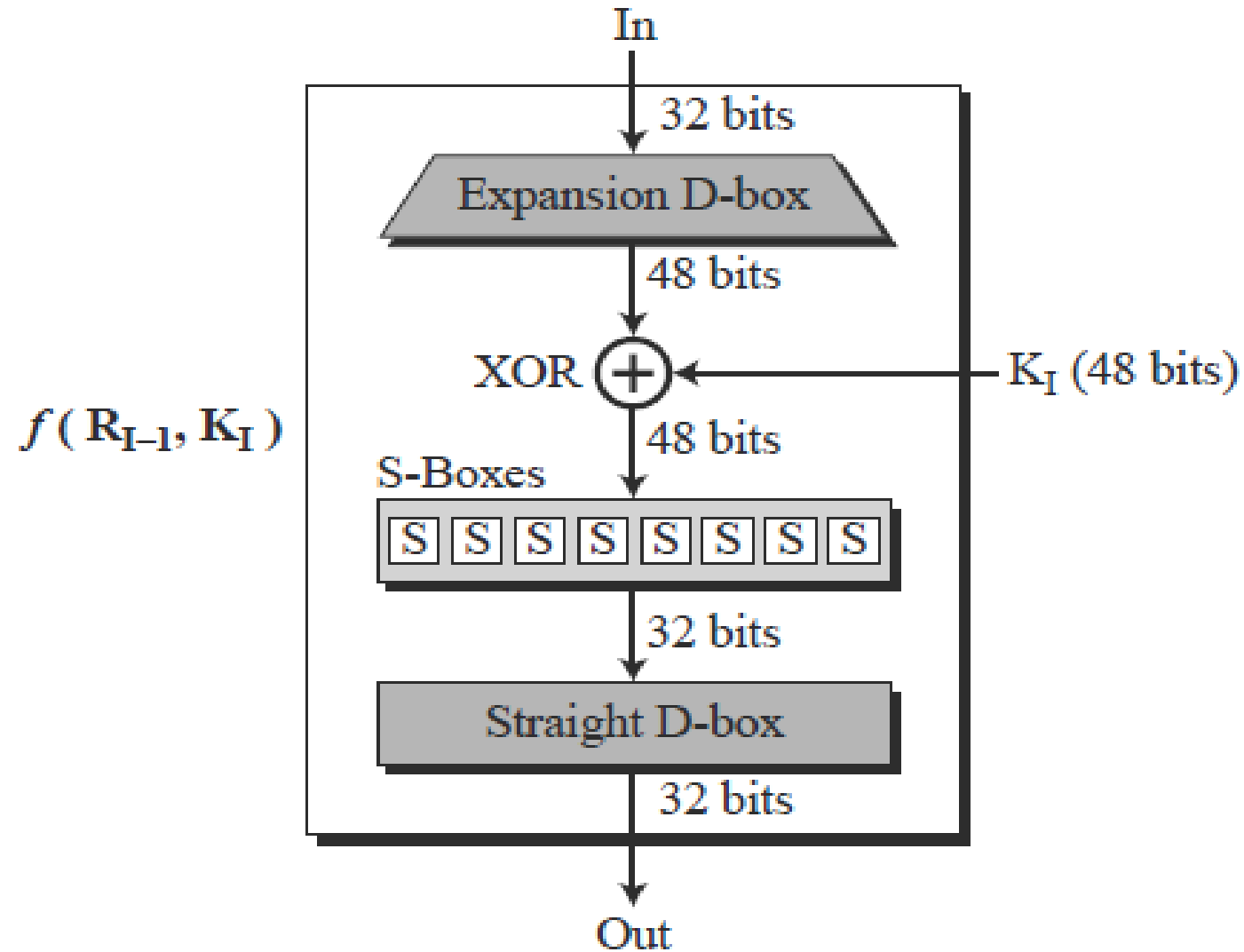
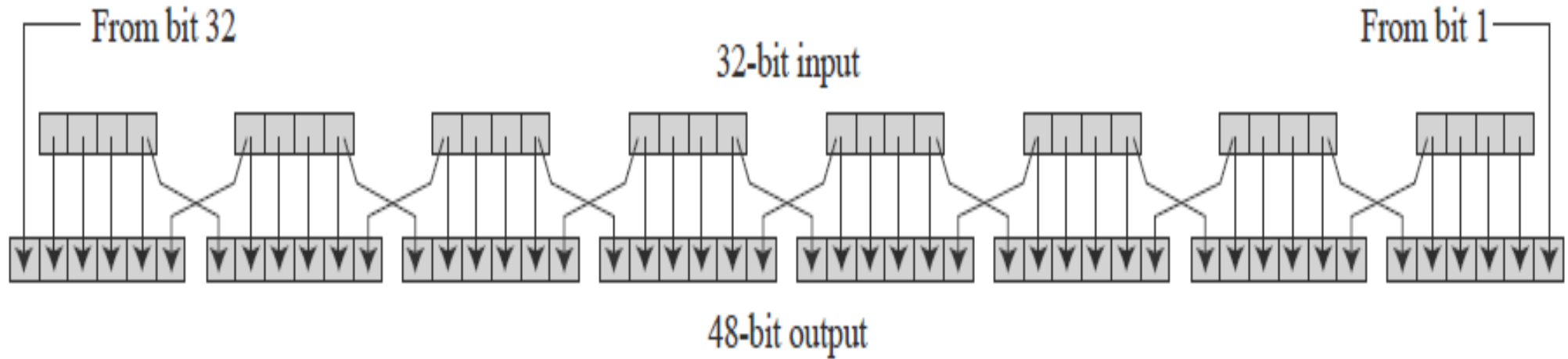


Figure 3.8 Single Round of DES Algorithm

- $LE_{16} = RE_{15}$
- $RE_{16} = LE_{15} + F(RE_{15}, K_{16})$
- Initial Permutation reorders the input data bits even bits to LH half, odd bits to RH half
- **Function F provides Confusion in Feistel**
 - eg. 8 bytes = 64 bits
 - $IP(675a6967\ 5e5a6b5a) = (ffb2194d\ 004df6fb)$



Expansion Permutation



E					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Substitution Boxes S

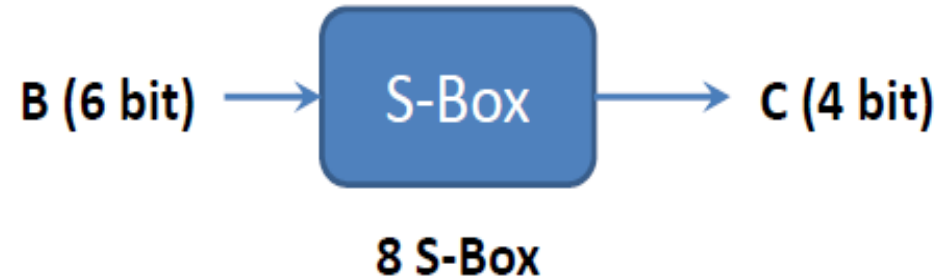
- have eight S-boxes which map 6 to 4 bits
- each S-box is actually 4 little 4 bit boxes
 - outer bits 1 & 6 (**row** bits) select one row of 4
 - inner bits 2-5 (**col** bits) are substituted
 - result is 8 lots of 4 bits, or 32 bits
- row selection depends on both data & key
 - feature known as autoclaving (autokeying)
- example:
 - $S(011001) = \text{row } 1, \text{column } 1100 (12)$
 - $= 1001 (9)$

S-box S_1

S_1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	01	10	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

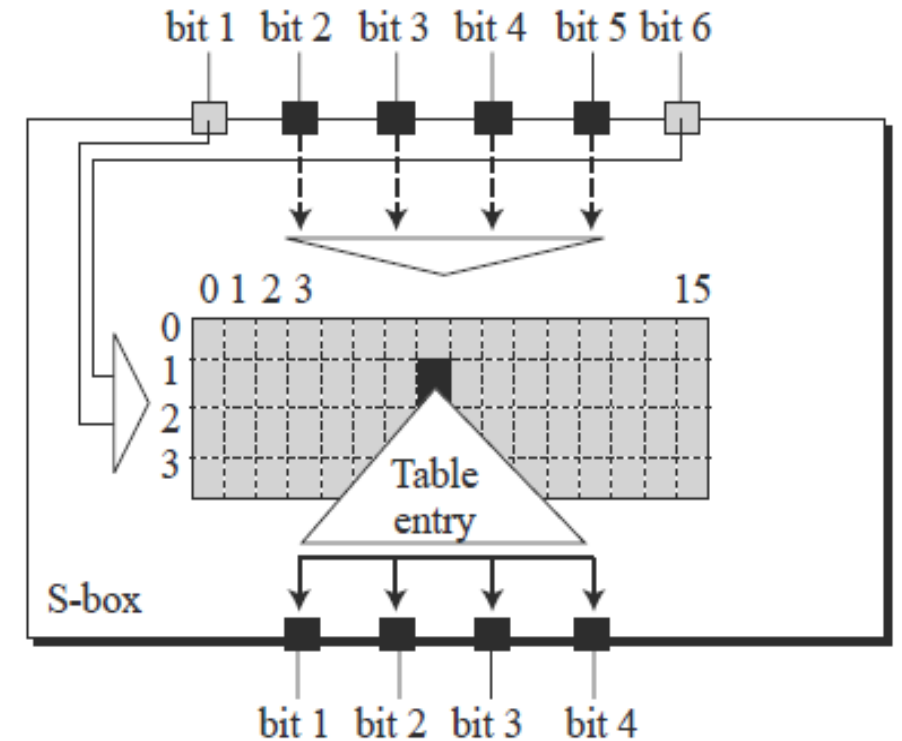
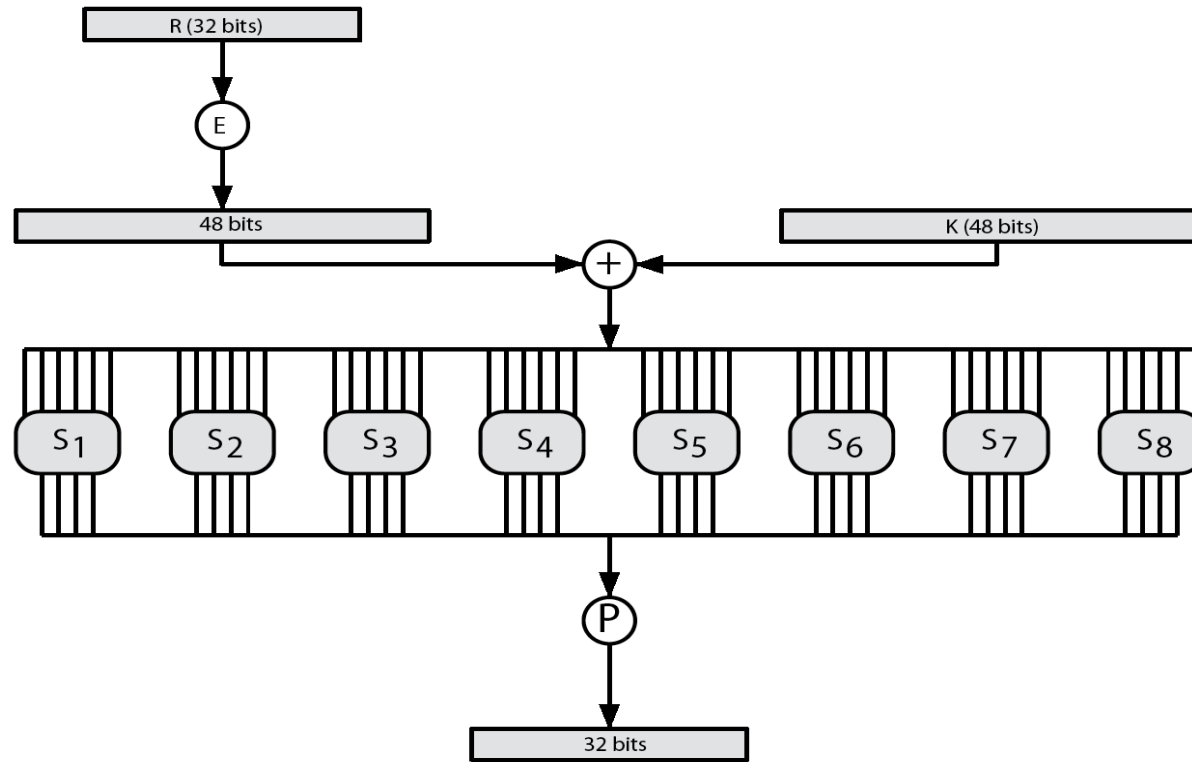
Substitution Boxes S

Each of the unique selection functions $S_1, S_2, \dots, S_8$, takes a 6-bit block as input and yields a 4-bit block as output



- S = matrix 4×16 , values from 0 to 15
- B (6 bit long) = $b_1 b_2 b_3 b_4 b_5 b_6$
 - $b_1 b_6 \rightarrow r$ = row of the matrix (2 bits: 0,1,2,3)
 - $b_2 b_3 b_4 b_5 \rightarrow c$ = column of the matrix (4 bits: 0,1,...15)
- C (4 bit long) = Binary representation of $S(r, c)$

Substitution Boxes S



Substitution Boxes S

שורה	מס' עמודה															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S₁																
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	3	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	13	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S₂																
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S₃																
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S₄																
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S₅																
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S₆																
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S₇																
0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S₈																
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Each S-box is actually 4 little 4 bit boxes

- outer bits 1 & 6 (**row** bits) select one row of 4 ,inner bits 2-5 (**col** bits) are substituted result is 8 lots of 4 bits, or 32 bits

example:

- $S(011001) = \text{row } 1,$
- $\& \text{ column } 1100(12)$
- $= 1001(9)$

Example (S1)

Row #	S_1	1	2	3	...											15	Column #
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7	
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8	
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0	
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13	

$S(i, j) < 16$, can be represented with 4 bits

Example: $B = 101111$

$$b_1b_6 = 11 = \text{row } 3$$

$$b_2b_3b_4b_5 = 0111 = \text{column } 7$$



$$C = 7 = \underline{0111}$$

Another example: $B = 011011$, $C = ?$

18

Permutation Function

- The last operation in the DES function is a permutation with a 32-bit input and a 32-bit output.
- The input/output relationship for this operation is shown in Table and follows the same general rule as previous tables.
- For example, the seventh bit of the input becomes the second bit of the output.

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

DES Key Schedule

- Forms subkeys used in each round
 - Initial permutation of the key (PC1) which selects 56-bits in two 28-bit halves
 - 16 stages consisting of:
 - rotating **each half** separately either 1 or 2 places depending on the **key rotation schedule K**
 - selecting 24-bits from each half & permuting them by PC2 for use in round function F
- note practical use issues in h/w vs s/w

DES Key Schedule

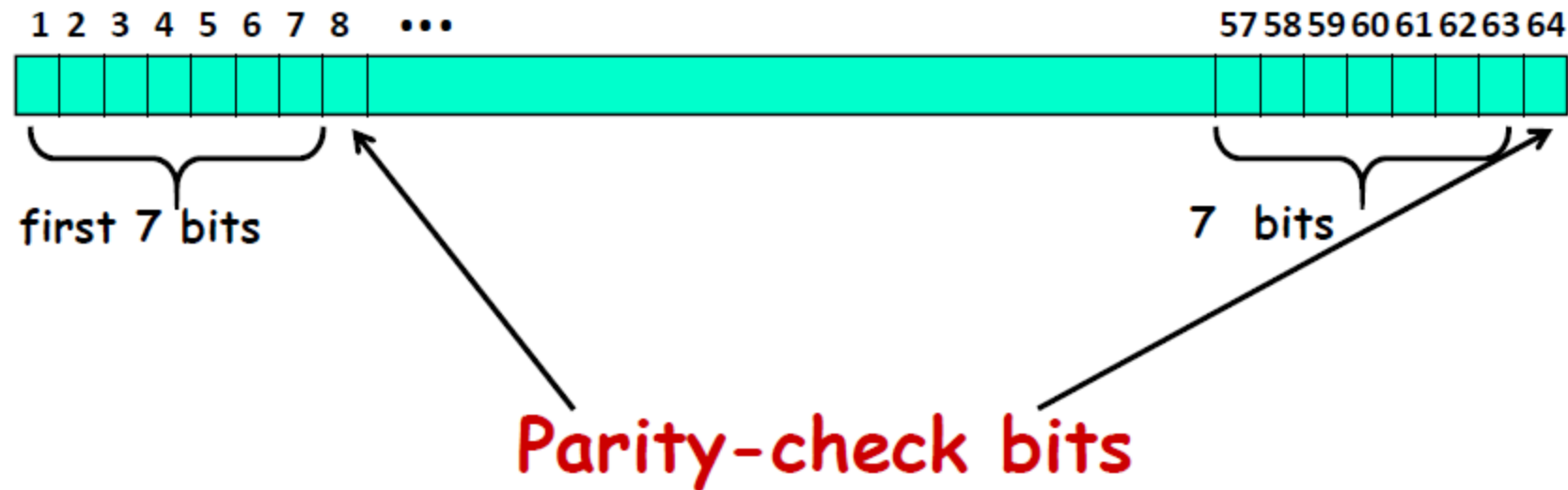
Initial Key size is 64 bits.

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

Input Key

Key length in DES

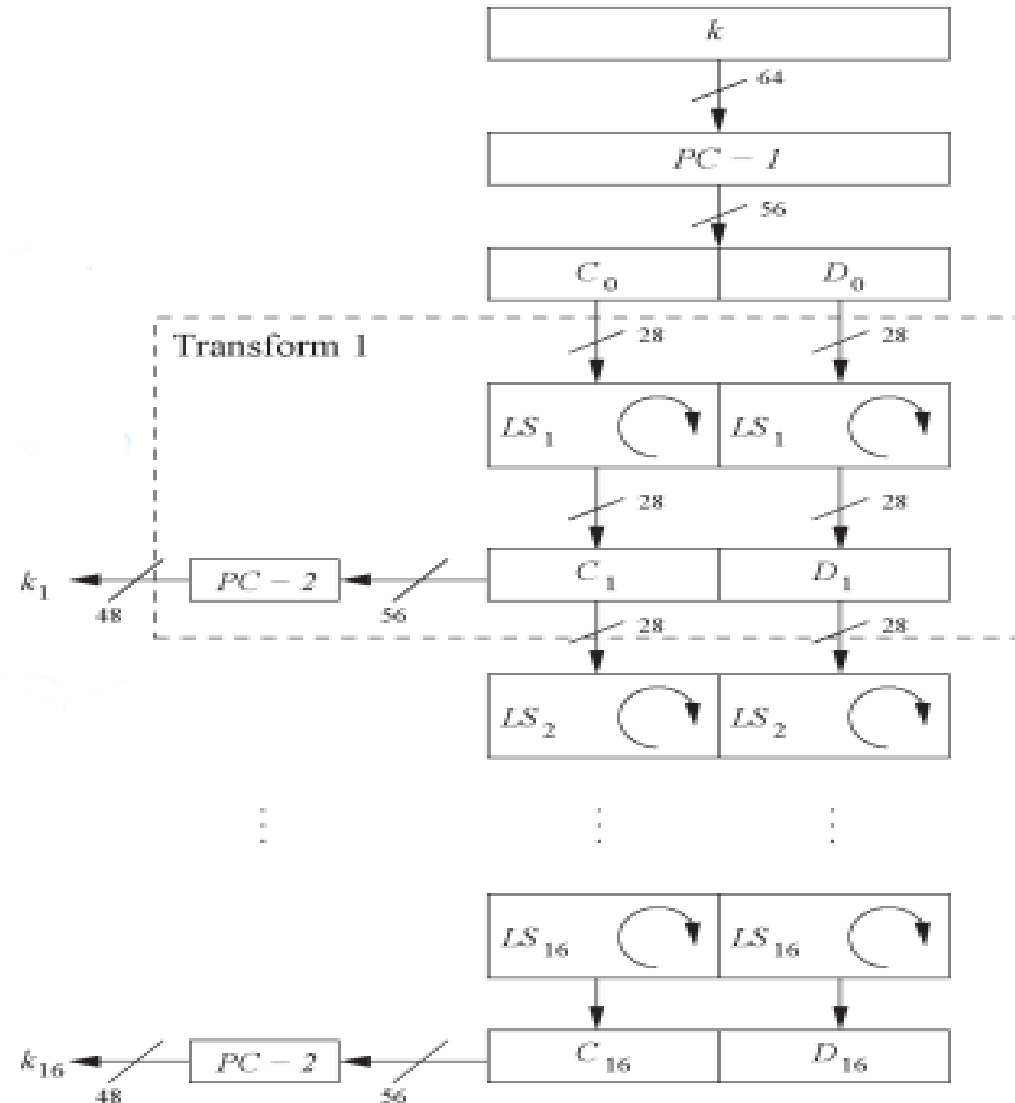
- In the DES specification, the key length is 64 bit:
- 8 bytes; in each byte, the 8th bit is a parity-check bit



Each parity-check bit is the XOR of the previous 7 bits

DES Key Schedule

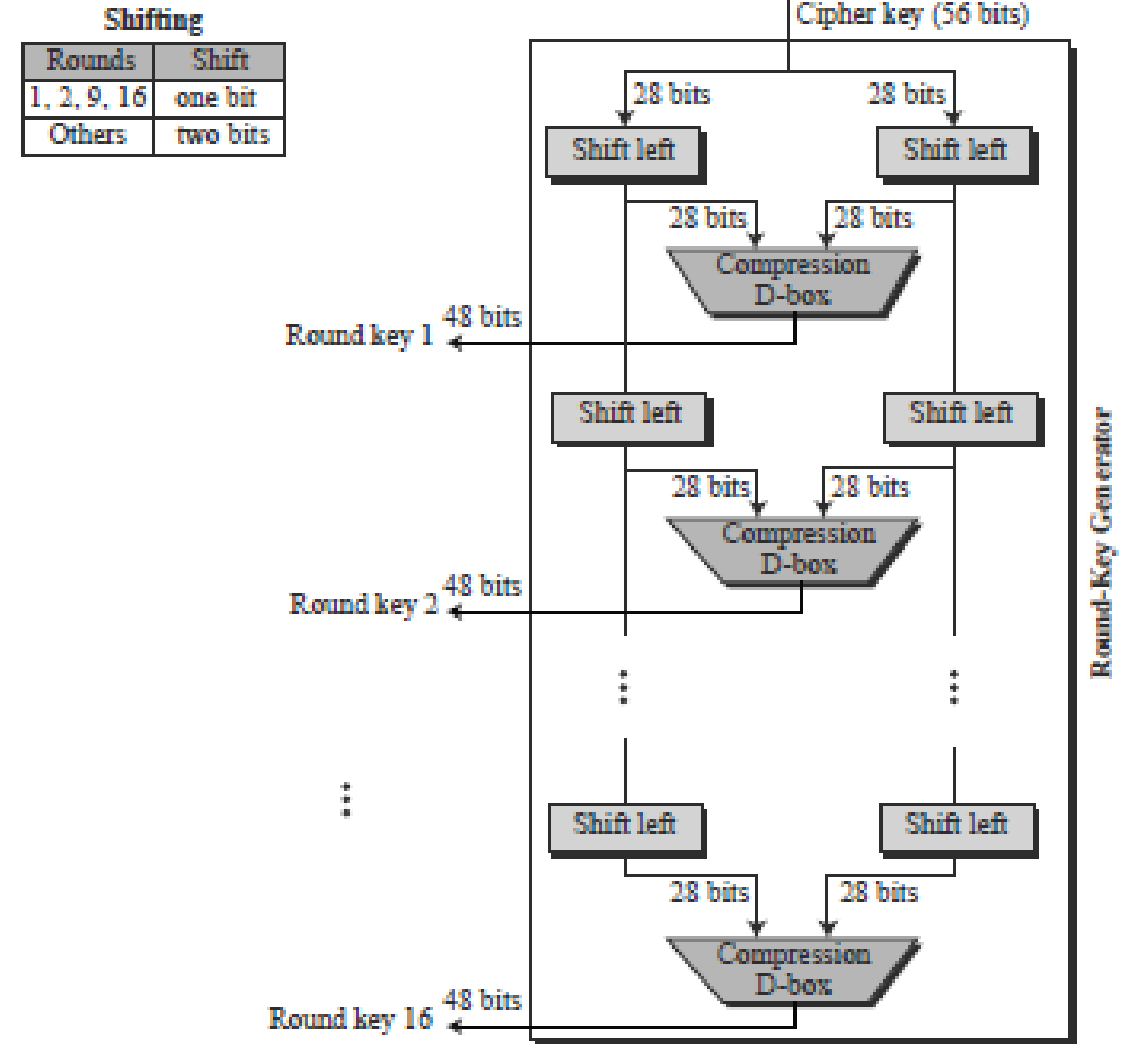
$PC - 2$							
14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32



Parity Drop: It drops the parity bits (bits 8, 16, 24, 32, ..., 64) from the 64-bit key and permutes the rest of the bits. The remaining 56-bit value is the actual cipher key which is used to generate round keys.

In rounds 1, 2, 9, and 16, shifting is one bit; in the other rounds, it is two bits.

The compression D-box changes the 58 bits to 48 bits, which are used as a key for a round.



Permutation Choice One (PC -1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38 ₀	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Schedule of Left Shifts

Round number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

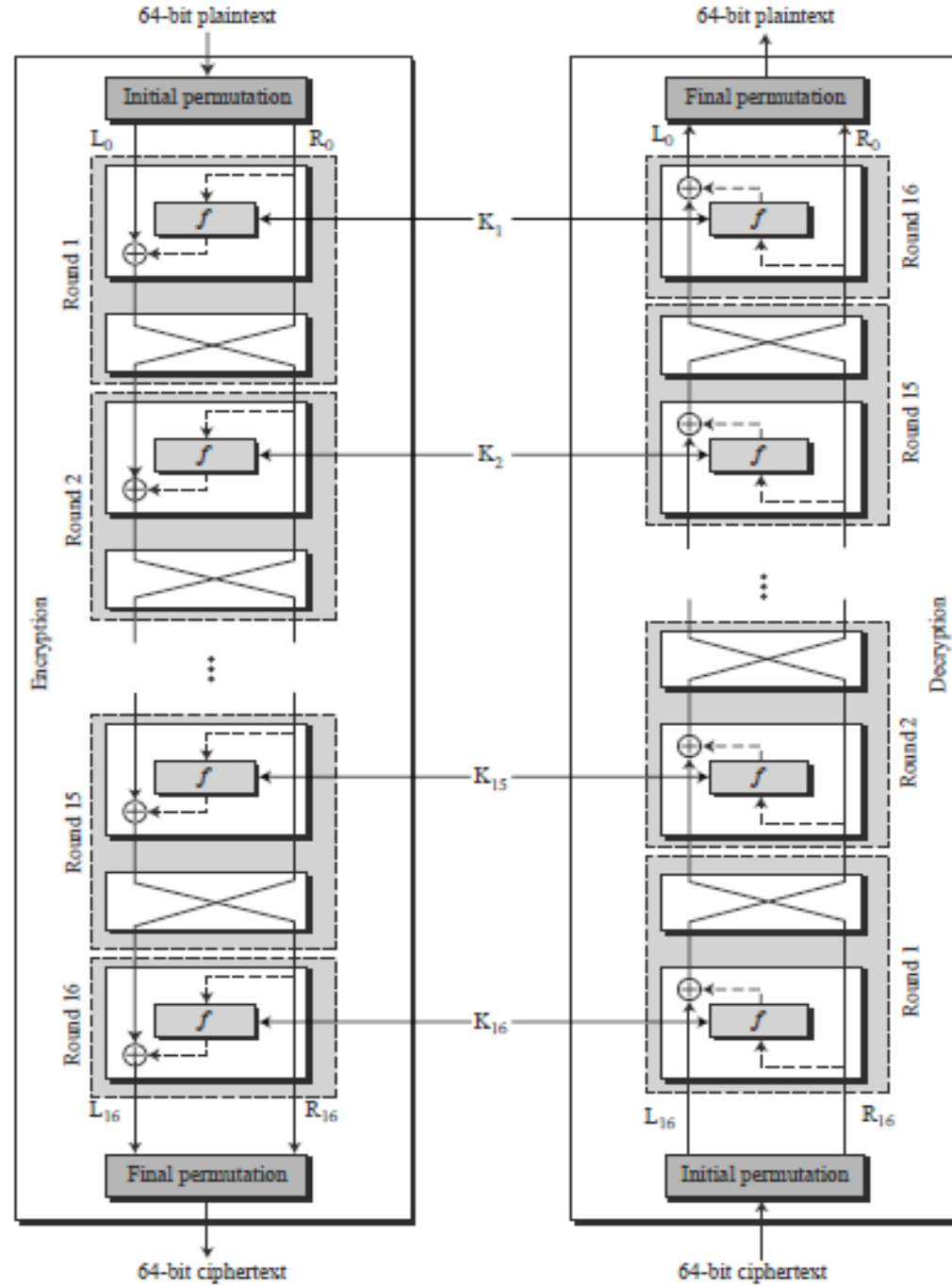
Permutation Choice Two (PC -2)

<i>PC – 2</i>							
14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Permutation Choice Two (*PC – 2*)

DES Decryption

- with Feistel design, do encryption steps again using subkeys in reverse order (SK16 ... SK1)
 - IP undoes final FP step of encryption
 - 1st round with SK16 undoes 16th encrypt round
 -
 - 16th round with SK1 undoes 1st encrypt round
 - then final FP undoes initial encryption IP
 - thus recovering original data value



DES

- Block size should be a compromise between size and speed
- Key size is directly proportional to strength of encryption
- Higher the number of rounds, tougher it is to crack
- Round function should be complex
- Speed is essential to handle large amounts of data

Avalanche Effect

- key **desirable property** of encryption alg
- where a change of **one** input or key bit results in changing approx **half** output bits
- DES exhibits strong avalanche

Average Time Required for Exhaustive Key Search

Key Size (bits)	Cipher	Number of Alternative Keys	Time Required at 10^9 Decryptions/s	Time Required at 10^{13} Decryptions/s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	2^{55} ns = 1.125 years	1 hour
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	2^{127} ns = 5.3×10^{21} years	5.3×10^{17} years
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	2^{167} ns = 5.8×10^{33} years	5.8×10^{29} years
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	2^{191} ns = 9.8×10^{40} years	9.8×10^{36} years
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	2^{255} ns = 1.8×10^{60} years	1.8×10^{56} years
26 characters (permutation)	Monoalphabetic	$2! = 4 \times 10^{26}$	2×10^{26} ns = 6.3×10^9 years	6.3×10^6 years

Thank You